

Search based approach to forecasting QoS attributes of web services using genetic programming



Yong-Yi Fanjiang^{a,*}, Yang Syu^b, Jong-Yih Kuo^c

^a Department of Computer Science and Information Engineering, Fu Jen Catholic University, New Taipei City 24205, Taiwan (R.O.C.)

^b Department of Computer Science and Information Engineering, National Taipei University of Technology, Taipei 106, Taiwan (R.O.C.)

^c Department of Computer Science and Information Engineering, National Taipei University of Technology, Taipei 106, Taiwan (R.O.C.)

ARTICLE INFO

Article history:

Received 22 December 2015

Revised 17 July 2016

Accepted 28 August 2016

Keywords:

Web service

QoS attribute forecasting

Genetic programming

Search-based software engineering

ABSTRACT

Context: Currently, many service operations performed in service-oriented software engineering (SOSE) such as service composition and discovery depend heavily on Quality of Service (QoS). Due to factors such as varying loads, the real value of some dynamic QoS attributes (e.g., response time and availability) changes over time. However, most of the existing QoS-based studies and approaches do not consider such changes; instead, they are assumed to rely on the unrealistic and static QoS information provided by service providers, which may seriously impair their outcomes.

Objective: To predict dynamic QoS values, the objective is to devise an approach that can generate a predictor to perform QoS forecasting based on past QoS observations.

Method: We use genetic programming (GP), which is a type of evolutionary computing used in search-based software engineering (SBSE), to forecast the QoS attributes of web services. In our proposed approach, GP is used to search and evolve expression-based, one-step-ahead QoS predictors. To evaluate the performance (accuracy) of our GP-based approach, we also implement most current time series forecasting methods; a comparison between our approach and these other methods is discussed in the context of real-world QoS data.

Results: Compared with common time series forecasting methods, our approach is found to be the most suitable and stable solution for the defined QoS forecasting problem. In addition to the numerical results of the experiments, we also analyze and provide detailed descriptions of the advantages and benefits of using GP to perform QoS forecasting. Additionally, possible validity threats using the GP approach and its validity for SBSE are discussed and evaluated.

Conclusions: This paper thoroughly and completely demonstrates that under a realistic situation (with real-world QoS data), the proposed GP-based QoS forecasting approach provides effective, efficient, and accurate forecasting and can be considered as an instance of SBSE.

© 2016 Elsevier B.V. All rights reserved.

1. Introduction

Recently, developing diverse software applications by composing existing services using *Service-Oriented Architecture* (SOA) has become both popular and prevalent. This software development approach is sometimes called *Service Composition* (SC) [1]. Currently, the most popular and common type of service used in SC is *Web Services* (WSs) [2–5] due to their numerous advantages, including interoperability, reusability, and universal accessibility. In industry, in concert with commonly accepted process definition languages such as the *Business Process Execution Language*

(BPEL) [6], integrating services to engineer software applications is already a mature technique. However, in academics, the focus has been on automating the manual composition process to widen the benefits offered by SC, a technique called *Automated Service Composition* (ASC). Although ASC is still an ongoing and fervid research area, we can view SC as a valid *Software Engineering* (SE) approach/branch. Because it centers on and is based on services, it is also called *Service-Oriented Software Engineering* (SOSE).

The design-time phases of SOSE/SC typically include the following: (1) the definition of an abstract workflow consisting of multiple abstract activities and a process structure, (2) discovering a set of functionality-equivalent concrete services on the Web or in service registries for each abstract activity, and (3) selecting the most suitable service to implement the activity for each abstract activity and in its corresponding service set. Readers can find more detailed descriptions and illustrations about the SOSE/SC process in

* Corresponding author.

E-mail addresses: yyfanj@csie.fju.edu.tw (Y.-Y. Fanjiang), a29066049@gmail.com (Y. Syu), jykuo@ntut.edu.tw (J.-Y. Kuo).

Section 3.1 of [7], Section 2 of [8], and Section III.D of [9]. The reviewed research on ASC proposes various approaches for automating the tasks and relieving the human labor force [1,10–15].

In service selection, when choosing the most suitable service from a set of functionality-equivalent services for an abstract activity, there are often many different nonfunctional service properties to be considered (selection criteria), such as the criteria reviewed in [1,10,11]. The criteria that should be used depends on user requirements and the specific application domain. However, in both industry and research, the most widely and intensively studied nonfunctional property has been *Quality of Service* (QoS), which has already been comprehensively studied in this research area. In addition, except for service selection, QoS also plays a crucial role in many other SOSE activities and phases such as QoS-based service discovery [16], classification [17], and ranking. Although QoS has been profoundly studied, several problems remain (e.g., how to semantically describe [18] and dynamically monitor [19] QoS), and one of the most critical and urgent issues is QoS's ability to change dynamically (e.g., QoS variation over time, which is the main focus and motivation of this study).

We identified two factors that can cause the actual QoS value of a service (WS) to vary, namely, *different service invocation times* and *disparate service consumers* (different service consumer locations) [1]. In this study, we concentrate on the former, i.e., changes in the QoS value over time. Section 2 demonstrates that QoS values vary over time, even for a single service. An intuitive way to address QoS variation over time and know the future value of QoS is by forecasting. Several QoS forecasting approaches have already been proposed. These approaches are introduced, analyzed, and compared in Section 3. In contrast to existing approaches, which primarily employ conventional time series forecasting methods/techniques, we propose using *Genetic Programming* (GP), which is a variant of the *Genetic Algorithm* (GA) and is a type of evolutionary computing used in *Search-Based Software Engineering* (SBSE) as reviewed and discussed in [20]. GP has already been applied to many different forecasting problems and domains; compared with other conventional forecasting methods/techniques, GP has demonstrated great forecasting performance (accuracy). The advantages of applying GP to this problem from both theoretical and practical application viewpoints and its performance and accuracy are discussed in Section 7.

To address the variation of QoS over time, current efforts primarily employ conventional time series techniques (a list and brief introduction to existing time series forecasting methods can be found in Section 2 of [21]) to find a predictor model and then use the identified model to forecast QoS values. In this paper, we use GP to search and evolve expression-based predictor models that can forecast QoS values. We propose using GP because of its superior performance (accuracy) in other time series forecasting applications, its flexibility in the form of the searched and generated predictors, and several other advantages that will be discussed later in this paper; however, one cost of using GP is that it is time-consuming. This paper is a major extension of our previous conference work presented in [22]. In this version, for each part of the study, more content, additional explanations, and more detailed descriptions are provided. Notably, one major difference between this paper and its previous version is that this paper introduces and implements more than ten different conventional time series forecasting methods; a comparison between these other methods and our GP-based forecasting approach is discussed, emphasizing its advantages and benefits. Below, we list and briefly discuss the contributions of this paper.

- First, in terms of the SBSE [20,23], we report the application of one of its methods, i.e., genetic programming, to a problem in SOSE (i.e., the QoS forecasting problem). In [23], the

authors proposed the concepts related to SBSE, and in [20], they exhaustively reported on existing SBSE instances (applications/cases). In both articles, the authors called for more applications/instances of metaheuristic search-based methods to diverse software engineering related problems and topics.

- Second, for the defined QoS forecasting problem, we also implement most of the conventional time series forecasting methods; a comparison between these methods and our proposed GP-based approach is discussed. The comparison of our GP-based approach with the conventional time series forecasting methods is a component of our approach's SBSE validity evaluation. We demonstrate our approach's validity as an SBSE application/case by performing the SBSE validity evaluation [23].
- Third, in this paper, we formally define the dynamic QoS forecasting problem so that other studies can develop their own approaches and compare them with our method or between each other.
- Finally, the proposed approach can be integrated with various QoS-based approaches used in SOSE and can enhance their reliability by offering more realistic reference QoS information. This concept is similar to the study conducted in [24,25], in which the authors developed their own QoS forecasting approach to provide QoS information for an existing ASC approach.

Fig. 1 shows how the proposed GP-based approach introduced in Section 5 works under the specified QoS forecasting problem, which is defined in detail in Section 4 of this paper. In the first phase (Phase 1), using past QoS observations as training data, GP is used to evolve and generate a valid expression-based QoS predictor. Subsequently, with a set of predictor inputs, the evolved predictor is used in Phase 2 to produce QoS forecasts. Finally, using the adopted forecasting accuracy measures, the forecasting performance (accuracy) of the proposed approach (its forecasting results) is calculated and then presented. The specified problem (including its three phases and the adopted measures) and the proposed approach are exhaustively described in Section 4 and Section 5, respectively.

The remainder of this paper is organized as follows. Section 2 demonstrates how QoS values of a service can vary and explains our research motivation. Existing QoS forecasting approaches and SBSE research are shown in Section 3. Section 4 formally defines the specifications of the problem to be solved and the adopted performance measures. Section 5 introduces and explains the core part of our proposed forecasting approach. Section 6 defines and briefly describes the time series forecasting methods used for comparisons with our method. The experimental results and other detailed discussions are presented in Section 7. Finally, Section 8 concludes this paper.

2. Reasons for QoS variation and research motivation

This section contains two parts. In the first part, variations in QoS values are discussed. The usefulness of the developed prediction approach is depicted in the second part.

2.1. QoS variation

Currently, QoS attributes are commonly and generally classified into two types: *domain-dependent* and *domain-independent* attributes (in Section 2.2 of [18]). In this study, we focus on domain-independent QoS, which is more widely used and studied than domain-dependent QoS. We further divide domain-independent QoS into two groups: (1) QoS attributes with fixed real values, and (2) QoS attributes that vary with one of the two QoS variation factors. We simply define these as *static QoS* and *dynamic QoS*, respec-

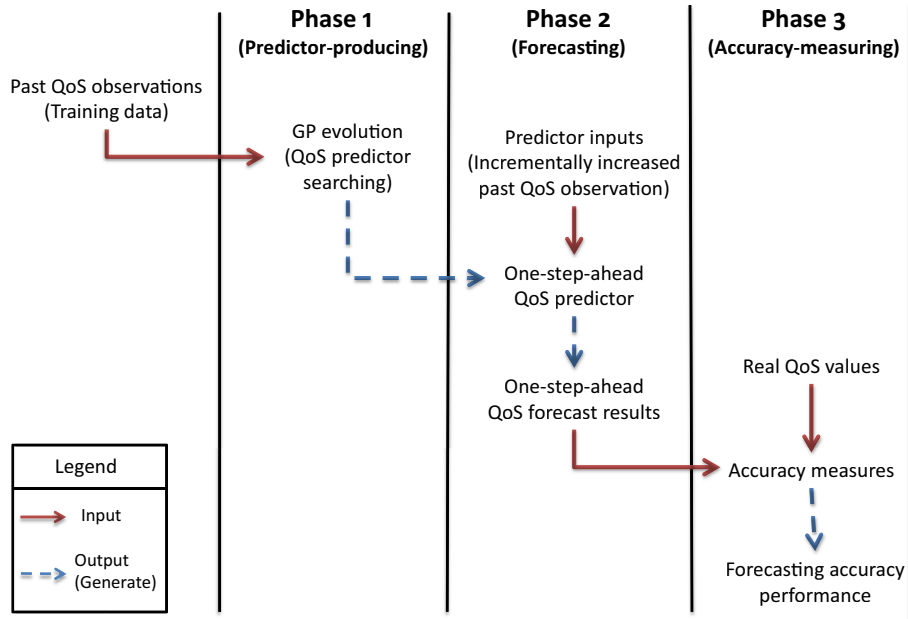


Fig. 1. A process-based general representation of how the proposed GP-based approach works in the specified QoS forecasting problem.

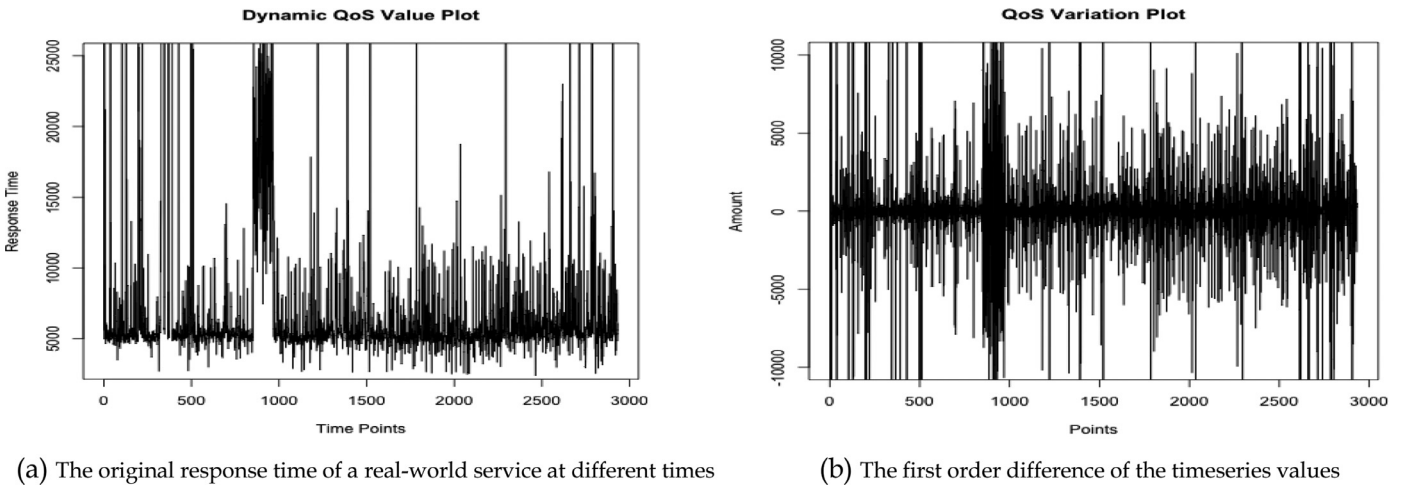


Fig. 2. The response time of a real-world service at different times.

tively. Obviously, the latter is the main focus of this study and of existing QoS prediction approaches.

There are two factors that can cause the QoS value to vary. First, because of heterogeneous user environments and network conditions, disparate service consumers primarily receive different dynamic QoS values. This can be verified using real-world large-scale QoS datasets such as the globally collected datasets presented in [26,27], in which some of the datasets can be viewed as two-dimensional matrices in which one dimension represents different service consumers and the other represents disparate services. To handle the prediction problem caused by variations among service consumers, there is already a series of research publications such as [28–31] that propose corresponding prediction approaches. Existing prediction approaches can be divided into two categories [31]: those based on *collaborative filtering* and those based on *matrix factorization*.

Secondly, dynamic QoS values can vary even when the same service is consumed by only a single service consumer. Fig. 2(a) shows a response-time series of a real-world WS recorded periodically and hourly at approximately 3000 different time points [32]. Fig. 2(b)

shows that the response time of the service fluctuates continuously over time and demonstrates the consecutive amount of change in the time series (i.e., the first-order difference of the timeseries, calculated by the formula $d_i = y_i - y_{i-1}$, where y_i is the original value of the timeseries at the time i and d_i is the difference between the value of two adjacent time points). Readers can find more illustrations and examples of real-world dynamic QoS variation over time in Section II of [33], Section 3.2 of [34], Section 4.2 of [35], Section 2 of [19], Section 1 of [25], Section 3.3 of [27], Section 8 of [36], and Section III.B of [37].

2.2. Research motivation

From the figures shown in the previous section, it can be seen that the static QoS data provided and published by service providers are neither trustworthy nor sufficiently precise for either human developers or automated QoS-based service approaches because dynamic QoS values can vary over time. There must be a reliable way to provide dynamic QoS attribute values that both human and automated approaches can refer to and rely on. Cur-

Table 1

A list of reviewed, existing QoS forecasting models.

The authors	The forecasting models used
Zadeh and Seyyedi [19]	ANNs
Senivongse and Wongsawangpanich [24]	ANNs
Z. Ye et al. [25]	ARIMA and Holts-Winters (Exponential smoothing)
B. Cavallo et al. [32]	Average, Last Observation, Linear Regression, and ARIMA
A. Amin et al. [33]	ARIMA-GARCH
A. Amin et al. [34]	ARIMA/SETARMA
M. Godse et al. [35]	ARMA
X. YunNi et al. [36]	ARMA
Z. Yilei et al. [37]	Tensor factorization
L. Mu et al. [45]	Structural equation modeling

rent existing approaches use *prediction* to address the problem. Another effective way to obtain real dynamic QoS values is via *service monitoring*; however, it is impossible to make proactive decisions or perform service operations [33,34] such as dynamic service selection and rebinding using only monitoring. In addition, service monitoring by continuous service invocations adds overhead to the monitored services and results in additional cost to users [19]. In contrast, by using prediction, human service engineers can use the results (predictors) to obtain likely future values of the dynamic QoS attributes of their candidate services and proactively perform their service actions. Moreover, the prediction approach can be integrated with various existing automated QoS-based service approaches such as service discovery [16], selection [15,38–41], and combination [42–44] approaches. In current prediction research for QoS variations over time, one published study focused on comparing the performance of various time series forecasting techniques, whereas several, including [19,33,34] and [37], concentrated only on proposing and developing a particular prediction approach. Still others [24,25,35,36,45] developed both a prediction approach and an automated service approach, which were combined and used together. From these references, it can be seen how the developed QoS forecasting approach may be used in or combined with service approaches such as automated selection/composition approaches. The authors concluded that providing QoS information concerning the two factors required for service designers to make service selections—composition and runtime late-binding—is essential.

3. Related work

In this section, we introduce and discuss related work. We divide this section into two parts: (1) the current QoS forecasting approaches and (2) search-based software engineering (SBSE).

3.1. Current approaches to QoS value prediction

Table 1 lists the reviewed approaches and the adopted forecasting models. In Section II of [21], the authors listed and classified current time series forecasting techniques into two groups: (1) classical methods, including *exponential smoothing* (comprising a *Holts-Winters* approach), *regression models*, *Autoregressive Integrated Moving Average* (ARIMA), *threshold*, and *Generalized Autoregressive Conditionally Heteroskedastic* (GARCH) methods, and (2) modern heuristic methods, including *Artificial Neural Network* (ANN)-based and *evolutionary computation*-based (comprising GA and GP) methods. As shown in Table 1, the most prevalent time series approach has been ARIMA/ARMA models. The author of [46] noted that *ARIMA models and exponential smoothing are the two most widely used approaches to time series forecasting*. The second most common approach has been ANNs. A technical introduction to these common time series models/techniques is presented in Section 6.

In the approaches based on ARIMA/ARMA models, some (i.e., [32,35,36]) follow and employ standard ARIMA models to make QoS predictions, whereas others make certain improvements (such as those described in [33] and [34]) or modifications (as in [25]) to the original ARIMA models. A. Amin et al. [33] combined both ARIMA and GARCH models to forecast QoS values because they showed that the dynamic QoS attributes of WSs can be characterized by high volatility and variation over time, which cannot be adequately modeled and handled by ARIMA models, although GARCH models were specifically designed for modeling and forecasting such time series data. In [34], the authors showed that the dynamic QoS attributes of WSs behaved nonlinearly in the real world—to the extent that current ARIMA models cannot guarantee accurate forecasting because they are based on a linearity assumption. When the incoming QoS data were found to be linear, their approach used the ARIMA model; otherwise, when the data were non-linear, they used the SETARMA model. Based on our reviews of the existing approaches listed in Table 1, most of them (except for [25]) assumed that the QoS forecasting problem is a case of *univariate time series forecasting*, meaning that when making predictions, the only available information is past observations of the particular QoS attribute to be predicted. In contrast, the authors of [25] found that the information contained in other past QoS observations (other dynamic QoS attributes) from the same service were helpful and should be utilized; thus, they modified the original univariate ARIMA model and the Holts-Winters approach into a *multivariate* model and approach, demonstrating that the modification improved the accuracy of the forecasting results.

B. Cavallo et al. [32] concentrated on comparing the performance of several time series forecasting methods, including ARIMA models, on QoS forecasting. The authors of [19] and [24] utilized ANNs to solve the forecasting problem. The approach used by [37] and [45] was not one of the usual time series forecasting methods listed in Section II of [21]; instead, the problem defined and solved by [37] was a QoS estimation problem that simultaneously considered the two QoS changing factors. The authors of [37] modeled their problem as a three-dimensional matrix (tensor) and solved it by matrix factorization. The specification of the problem to be solved in [37] differed both from our approach and from the other studies listed in Table 1 because the authors did not consider and incorporate the second QoS changing factor (disparate service consumers). Structural equation modeling was used in [45]; because this technique is an unusual time series forecasting method, it was not incorporated in our comparison.

3.2. Search-based software engineering

The term *search-based software engineering* and the identification and a definition of SBSE were first presented in [23], in which the authors discussed the elements that should be considered when reformulating an SE problem as an SBSE problem, including its representation, fitness function, and operators. In [20], these same authors presented a comprehensive review and analysis of current methods used in SBSE such as Hill Climbing (HC), GP, GA, and Simulated Annealing (SA) in relation to the various SE problems and issues encountered for different SE aspects and phases such as requirements/specifications, design tools and techniques, and software/program verification. Their reviews and analyses were very thorough and complete. According to this review, in current SBSE research, GP has mainly been used for making or assisting in predictions such as cost, effort, defect, and quality estimations for developing software projects. However, based on our investigation and the review presented in [20], the use of GP has not yet been reported for forecasting dynamic QoS values of services/WSs.

4. Problem specification and measurements

In this section, we formally specify the problem to be solved in this paper (Section 4.1) and the performance measurements used to evaluate all of the devised and implemented approaches (Section 4.2).

4.1. Problem definition

The problem is formulated as a time series forecasting problem. We first denote T as a set of discrete time points for the observations in a time series (index set):

$$T = \{0, 1, \dots, k-1, k\} \quad (1)$$

where the cardinality (the number of the elements contained in the set) is $k+1$. Then, we let $Y_{0..k}$ represent a time series consisting of the observations (real QoS values) of a dynamic QoS attribute at the times contained in T :

$$Y_{0..k} = \{y_0, y_1, \dots, y_i, \dots, y_{k-1}, y_k\} \quad (2)$$

where the length of the time series is $k+1$, y_i represents the observations of the dynamic QoS attribute at time i , $i \in T$, and $y_i \in Y_{0..k}$. Subsequently, we let cur denote the current time (the present), $cur \in T$, and $y_{cur} \in Y_{0..k}$:

$$T = \{0, 1, \dots, cur, \dots, k-1, k\} \quad (3)$$

$$Y_{0..k} = \{y_0, y_1, \dots, y_{cur}, \dots, y_{k-1}, y_k\} \quad (4)$$

We assume that because cur represents the present, i.e., at time cur , the QoS observations contained in $Y_{0..cur}$ ($\{y_0, y_1, \dots, y_{cur}\}$) are already known and available, while the QoS observations within $Y_{cur+1..k}$ (i.e., $\{y_{cur+1}, y_{cur+2}, \dots, y_k\}$) are still unknown and must be forecasted (they are the targets to be predicted).

With the available resources $Y_{0..cur}$ (training data), the goal is to find a forecasting model (predictor) P capable of producing accurate predictions for QoS observations at the time points $cur+1..k$ (i.e., $Y_{cur+1..k}$). We let f_{cur+1} denote the prediction produced by P with the available data (the past observations) at time cur (i.e., $Y_{0..cur}$) as its prediction input:

$$f_{cur+1} = P(Y_{0..cur}) \quad (5)$$

where f_{cur+1} is a prediction of a QoS observation at time $cur+1$ and is generated by the found predictor P using the past QoS observations $Y_{0..cur}$ as training input. In this study, we concentrate on *one-step-ahead* forecasting; thus, to obtain the predictions for all subsequent QoS observations by the predictor P , we must forecast iteratively:

$$\begin{aligned} f_{cur+1} &= P(Y_{0..cur}) \\ f_{cur+2} &= P(Y_{0..cur+1}) \\ f_k &= P(Y_{0..k-1}) \end{aligned} \quad (6)$$

Under this defined and formulated forecasting process, as time evolves (i.e., cur , $cur+1$, $cur+2$, ...) the available number of past observations (predictor input) also increases. In this manner, we can obtain a set of forecasting results represented by $F_{cur+1..k}$:

$$F_{cur+1..k} = \{f_{cur+1}, f_{cur+2}, \dots, f_k\} \quad (7)$$

Specifically, in our problem context, any forecasting approach should experience two phases to have a set of predictions ($F_{cur+1..k}$): (1) a *model-producing* (predictor training/searching) phase and (2) a *forecasting* phase. In the first phase, a forecasting approach would employ the available resources, i.e., $Y_{0..cur}$, to search and construct a predictor. Then, in the second phase, the approach iteratively utilizes the predictor to forecast QoS values

(i.e., f_{cur+1} , f_{cur+2} , ...). Each forecasting method has its own predictor form (structure) and disparate procedure to set up and construct the predictor (e.g., parameter settings).

Finally, we define two symbols tl and pl to represent the size of the available resources (the cardinality of $Y_{0..cur}$, i.e., $tl = cur + 1$) and the length of the future forecast time period (the cardinality of $F_{cur+1..k}$, i.e., $pl = k - cur$), respectively.

4.2. Performance measurements

In this study, after the first and second phases, each forecasting method goes through a third phase, i.e., a *measuring* phase, where a measurement is made using two basic metrics, the *mean absolute error* (MAE) and the *mean absolute percentage error* (MAPE), to evaluate the forecasting performance (accuracy) of the implemented methods. We also define a comparison measurement, called the *Relative Prediction Accuracy Improvement* (RPAI), to compare the forecasting accuracy of two different methods and to determine the percentage of forecasting accuracy improvement offered by our GP-based approach. The definition of the MAE is

$$MAE = \frac{1}{pl} \sum_{i=1}^{pl} |y_i - f_i| \quad (8)$$

where pl is the length of the forecast time period (the number of time points) and y_i and f_i represent the real observation and the predicted value at time i , respectively. Because the MAE is an error-based measurement, lower values are better. The definition of MAPE is below.

$$MAPE = \frac{1}{pl} \sum_{i=1}^{pl} \frac{|y_i - f_i|}{y_i} \quad (9)$$

Because MAPEs are percentages, it is relatively easy to understand the accuracy represented by the MAPE provided by a forecasting method. The MAPE is also an error-based measurement. Finally, we define the RPAI as follows:

$$RPAI = \frac{(MAPE_{p1} \times 100) - (MAPE_{p2} \times 100)}{(MAPE_{p1} \times 100)} \quad (10)$$

where $MAPE_{p1}$ is the MAPE received from the first compared predictor, and $MAPE_{p2}$ represents the MAPE from the second compared predictor. Based on the MAPE, there are three possible cases that can occur: the forecasting accuracy of the first predictor (i.e., $P1$) could be higher, lower, or equal to that of the second predictor ($P2$). We formally specify these three cases below:

$$\begin{cases} P1 >_{\text{accurate}} P2, & RPAI \text{ value} < 0 \\ P2 >_{\text{accurate}} P1, & RPAI \text{ value} > 0 \\ P2 =_{\text{accurate}} P1, & RPAI \text{ value} = 0 \end{cases} \quad (11)$$

where $>_{\text{accurate}}$ means that the first predictor (the left operand) is more accurate than the second (the right operand) in forecasting and $=_{\text{accurate}}$ means that the forecasting accuracy of the two compared predictors is identical.

In addition to comparing the predictors according to two different methods, we also utilize the MAPE to measure the amount of internal forecasting accuracy improvement via our GP-based approach. In our approach, each GP generation holds a set of possible solutions (predictors), one of which is the best solution (a solution with the best fitness value) in that generation. An evolutionary approach uses multiple generations to evolve and discover a set of best or near-optimal, acceptable answers (solutions). Using this problem-solving pattern, we can determine whether the best predictors' forecasting accuracy improves after a GP evolutionary cycle. For this purpose, $RMAE_{p1}$ represents the MAPE of the best predictor found in the first GP generation, and $RMAE_{p2}$ is the MAPE of the best predictor found in the last generation. In our defined problem, there are three different phases that each forecasting method must go through.

Table 2

A GP evolution process presented in functional pseudo code.

Input: A set of GP parameters, including <i>g</i>: the number of generations in a GP evolution <i>p</i>: the number of chromosomes contained in each generation's GP population <i>cr</i> and <i>mr</i>: crossover and mutation rates (probabilities) <i>fs</i> and <i>ts</i>: function and terminal sets. <i>ff</i>: the fitness function adopted in the evolution Output: <i>b</i>: the best chromosome from the last evolved generation Variables: <i>Population</i>: the current generation's population containing a set of chromosomes <i>CurrentGenerationNumber</i>: current generation number <i>NewPopulation</i>: next generation's population <i>Parent1</i> (<i>Parent2</i>): a parent chromosome selected by the selection operator <pre> 1. <i>Population</i> ← <i>Initialize</i>(<i>fs</i>, <i>ts</i>); 2. while(<i>CurrentGenerationNumber</i> < <i>g</i>) 3. <i>Evaluate</i>(<i>Population</i>, <i>ff</i>); 4. <i>NewPopulation</i> ← null; 5. while(<i>Size</i>(<i>NewPopulation</i>) < <i>p</i>) 6. <i>Parent1</i> ← <i>Select</i>(<i>Population</i>); 7. <i>Parent2</i> ← <i>Select</i>(<i>Population</i>); 8. if(<i>Random</i>() < <i>cr</i>) 9. <i>Crossover</i>(<i>Parent1</i>, <i>Parent2</i>); 10. if(<i>Random</i>() < <i>mr</i>) 11. <i>Mutate</i>(<i>Parent1</i>); 12. if(<i>Random</i>() < <i>mr</i>) 13. <i>Mutate</i>(<i>Parent2</i>); 14. <i>Push</i>(<i>Parent1</i>, <i>Parent2</i>, <i>NewPopulation</i>); 15. <i>Population</i> ← <i>NewPopulation</i>; 16. <i>CurrentGenerationNumber</i>++; 17. return <i>GetBest</i>(<i>Population</i>); </pre>	Functions: <i>Initialize</i>(<i>fs</i>, <i>ts</i>): randomly generate the first generation's population (chromosomes) based on a specified function (<i>fs</i>) and terminal (<i>ts</i>) sets and return it <i>Evaluate</i>(<i>Population</i>, <i>ff</i>): calculate a fitness value for each chromosome contained in <i>Population</i> based on a fitness function (<i>ff</i>). <i>Size</i>(<i>Population</i>): return the size of the assigned <i>Population</i> (i.e., the number of chromosomes contained in <i>Population</i>) <i>Select</i>(<i>Population</i>): select a chromosome from <i>Population</i> and return it (selection operator) <i>Random</i>(): return a random value between 1 and 0 <i>Crossover</i>(<i>chromosome1</i>, <i>chromosome2</i>): crossover two incoming chromosomes (crossover operator) <i>Mutate</i>(<i>chromosome</i>): mutate <i>chromosome</i> (mutation operator) <i>Push</i>(<i>chromosome1</i>, <i>chromosome2</i>, <i>Population</i>): put <i>chromosome1</i> and <i>chromosome2</i> into <i>Population</i> <i>GetBest</i>(<i>Population</i>): return the best chromosome in <i>Population</i>
--	--

- (1) *Model- (predictor-) producing phase*. In this phase, using past QoS observations (training data), i.e., $Y_{0...cur}$, each forecasting approach must generate a valid one-step-ahead predictor P .
- (2) *Forecasting phase*. Here, a set of one-step-ahead forecast results, i.e., $F_{cur+1...k}$, is generated by P with the incrementally increased past QoS observations (i.e., $Y_{0...cur}$, $Y_{0...cur+1}$, $Y_{0...cur+2}$, ...) as the predictor's inputs.
- (3) *Accuracy-measuring phase*. In this phase, using the set of forecast results ($F_{cur+1...k}$) and a set of corresponding real QoS values (the training data, i.e., $Y_{cur+1...k}$), the MAE and MAPE are applied to calculate a quantified forecasting accuracy. Otherwise, the RPAI is used in this phase to compare two disparate forecasting approaches and study our proposed approach's internal forecasting improvement.

5. The genetic programming based QoS forecasting approach

Our approach exploits GP, which is a variant of Genetic Algorithms (GAs), to evolve and search for a one-step-ahead, expression-based QoS predictor. In other words, GP corresponds to and is used in the training stage (model/predictor-producing phase) of the problem formalized in Section 4. Detailed introduction to both GAs and GP can be found in [47].

5.1. GP evolution process

Table 2 shows a conventional GP evolution process in functional pseudo code. At the beginning (line 1), a GP generates a set of chromosomes (i.e., *population*) with a set of defined functions and

terminals (*fs* and *ts*). Next, the process enters a repetitive generation evolution (lines 3–16) that continues as long as the current generation number is lower than the total number of generations (*g*) allowed. When this condition is no longer satisfied, the GP process ends; otherwise, the process will iterate through a new generation. The chromosomes contained in the population (*Population*) are manipulated by selection (lines 6 and 7), crossover (lines 8 and 9), and mutation (lines 10–13) operators in sequence to construct a new population (*NewPopulation*) for each generation. In each generation, before the construction of the next generation's population (lines 5–14), each chromosome within the current population (*Population*) is evaluated by an adopted fitness function (*ff*) to give it a score representing the suitability of the chromosome (fitness) to the current problem (line 3). Subsequently, the GP process starts an inner loop (lines 5–14) to construct the next generation's population. The inner loop executes repetitively to add more chromosomes to the new population until the number of chromosomes in the next generation's population (*Size*(*NewPopulation*)) reaches the default population (*p*) size. At the beginning of the inner loop (lines 6 and 7), two chromosomes, called the *parents* in GP term, are picked up by running the selection operator twice. After the selections, the two parent chromosomes may undergo a crossover phase (line 9) based on a specified crossover probability (line 8). Subsequently, each parent may undergo mutation (lines 11 and 13) based on an assigned mutation probability (lines 10 and 12). Finally, the two parents are pushed into the new population (line 14). When there are sufficient new chromosomes to complete a new generation, the inner loop ends. Then, the current population is replaced by the new population (line 15), the current generation

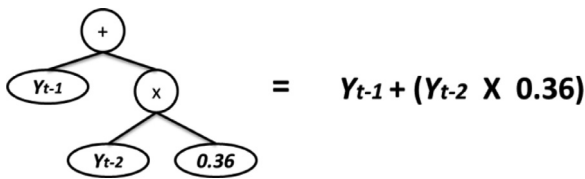


Fig. 3. An example of GP's binary tree-structured chromosome and its corresponding represented expressions.

number is increased by 1 (line 16), and finally, the GP evolutionary process proceeds to the next generation.

5.2. The elements involved in GP

This section introduces, explains, and illustrates the elements used and involved in a GP process.

5.2.1. Chromosomes

In GP, each chromosome, which represents a candidate solution to the solved problem, is a tree structure suitable for GP operations. If the problem were to discover a valid program, then each chromosome evolved and operated on by GP would be a program represented (encoded) in a tree form. However, because the purpose of this study is to determine mathematical expression-based QoS predictors, our GP's chromosomes are mathematical expressions encoded and represented in a binary tree, as shown in Fig. 3.

5.2.2. Function and terminal sets

Before running the GP, its operators must determine what elements should be included in its function and terminal sets. GP's function set determines what operators are allowed to appear in the tree-structured expressions (chromosomes). For example, in Fig. 3, it is obvious that its function set must include + and × operators. However, the possible expression operands are specified by and included in the GP's terminal set. As an example, in Fig. 3, the GP's specified function set must contain, at minimum, the expressions Y_{t-1} , Y_{t-2} , and a value between 0 and 1 (which may be a random variable).

5.2.3. Selection

The purpose of GP's selection operator is to take a chromosome from a set of chromosomes (a population) and let that chromosome act as a parent to perform two GP subsequent evolutionary operations (i.e., crossover and mutation). Because GP is an evolutionary problem-solving approach, its selection is not random; instead, chromosome selection is based on the fitness value attached to each chromosome and provided by a specified fitness function. Currently, there are two major selection operators (i.e., *binary tournament* and *roulette-wheel*) are commonly used in both GAs and GP. The former is a proportional method by which a chromosome with a better fitness value is more likely to be selected as a parent. The latter randomly chooses two chromosomes from the current population and then lets the winner (the chromosome with a better fitness value) become a parent. Fig. 4 illustrates how binary tournament selection works.

5.2.4. Crossover

The crossover operator exchanges one part (gene) of one parent chromosome with that of another parent chromosome. After the two selection operations are completed, the two selected parents have relatively better survivability (fitness) for the current problem. In GP, it is considered that fitter chromosomes (parents) also have better genes that make them more suitable to the problem. Thus, the purpose of the crossover operator is to exchange and

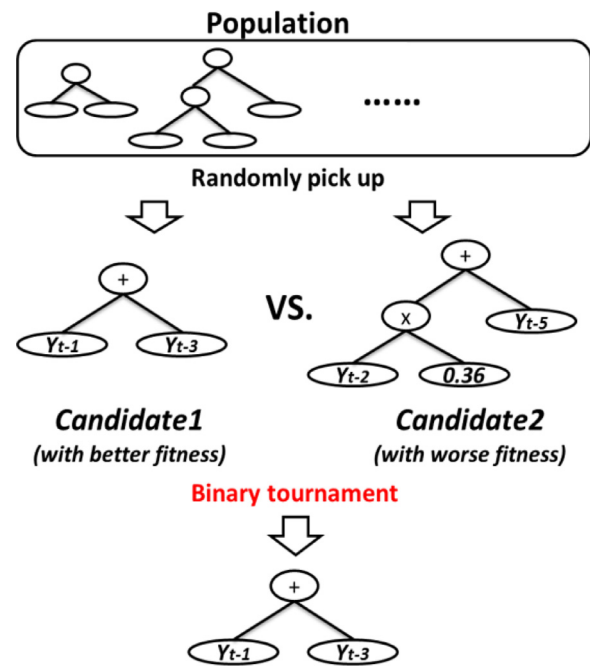


Fig. 4. An example of how GP's selection operator works (binary tournament).

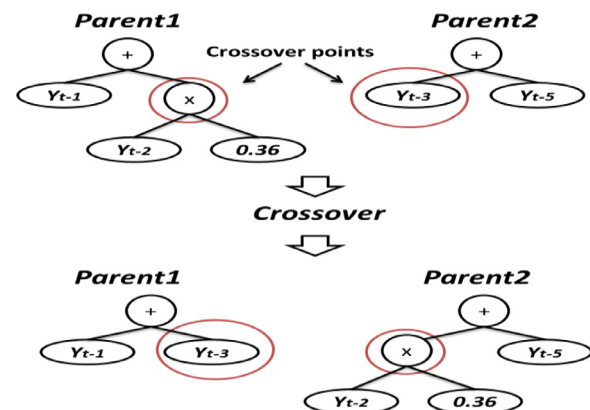


Fig. 5. An example of how GP's crossover operator works.

move these better genes between chromosomes (parents) to provide even better chromosomes in the next generation. Fig. 5 shows an example of crossover.

5.2.5. Mutation

In GP, mutation is implemented by randomly changing a node of a chromosome to a different node. Fig. 6 shows an example of mutation in GP. Mutation can enhance the diversity of the evolved population and improve GP's exploration of the search space. The probability of mutation is usually set far lower than the probability for crossover within a GP.

5.2.6. Fitness function

This is the most important and crucial component of any GP-based approach. In Section 2.2 of [23], the authors briefly noted the issues and principles that must be considered when defining and devising fitness functions. Beyond those, the reference sources for fitness function design are the measurements and metrics defined and used in a particular problem domain. For example, as noted in [23] and reviewed in [20], diverse proposed software metrics or their modifications can be used as fitness functions in SBSE research. As another example, there are several forecasting accuracy

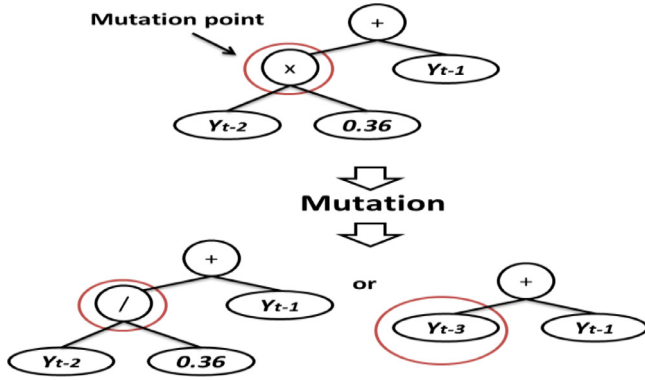


Fig. 6. Two examples of GP's chromosome mutation.

measures in time series forecasting, such as those introduced in Section 2.5 of [46] and those used by the approaches reviewed in our related work section. Therefore, it is both intuitive and sensible to use one of those measures as the fitness function for a time series forecasting problem.

In the past, GP has been applied to other forecasting problems. We can divide these into cross-sectional forecasting studies, such as [48] and those software estimation approaches reviewed in [20], and time series forecasting studies, such as [21,49–51]. An introduction to these two types can be found in Chapter 1.4 of [46]. Our GP-based approach belongs to the latter, i.e., time series forecasting. We formulated the forecasting problem as a time series forecasting problem in the previous section. GP has been reported as having better forecasting performance (accuracy) than other conventional time series forecasting techniques (for example, in [50], the authors used GP to forecast financial data and discovered that GP was better than ANNs for their particular forecasting problem), although the ability to forecast QoS values using GP has never been tested; thus, this is the first study to apply GP to a dynamic QoS forecasting problem.

6. The time series forecasting techniques for comparison

As defined in Section 4, the studied problem is a time series forecasting problem; thus, in this study, we also utilize and implement conventional time series forecasting methods and compare them with our approach to compare the relative forecasting performance (accuracy) of our approach (using the measurements defined in Section 4.2) and to verify our approach's validity as an SBSE. In this section, we introduce the time series forecasting methods that were implemented and compared with our approach; these methods fell into five groups: (1) benchmark methods, (2) regression methods, (3) machine learning approaches, (4) sophisticated methods, and (5) random searches.

6.1. Benchmark methods

6.1.1. Average method

In this method, the prediction result is always the mean of all available historical data. Its predictor model is as follows:

$$F_{cur+1} = \frac{1}{l} \sum_{i=0}^l y_i \quad (12)$$

where F_{cur+1} represents the forecast to a future observation at time $cur + 1$, l is the number of available historical data, and y_i is historical data from past time i .

6.1.2. Naïve method

This method is also called the last observation approach. Its forecasting result is simply the last available and known observation from the historical data:

$$F_{cur+1} = y_{cur} \quad (13)$$

where y_{cur} represents the last available and known observation.

6.1.3. Seasonal naïve method

This method is a variation of the naïve method that is most useful and suitable for highly seasonal time series. In this method, a forecasting result is set to be the same as an observation from the previous same season. For example, if data in a time series were collected hourly (i.e., there are 24 observations every day) and the data's seasonality is weekly, a weekly seasonal naïve forecast would be the past observation at the same day and hour of the previous week:

$$F_{cur+1} = y_{last_season} \quad (14)$$

where y_{last_season} denotes the time of the same previous season. In this study, two seasonal naïve methods, i.e., the weekly seasonal naïve method and the daily seasonal naïve method, are discussed.

6.1.4. Drift method

This method is also a variant of the naïve method. Its forecast is the result of the naïve method plus a drift, which is the average changing amount within the historical data:

$$F_{cur+1} = y_{cur} + \frac{1}{cur} \sum_{i=1}^{cur} y_i - y_{i-1} \quad (15)$$

where the first term is identical to the naïve method, whereas the second term is used to calculate the drift amount.

A more detailed introduction to the above benchmark methods can be found in Chapter 2.3 of [46].

6.2. Regression methods

In statistics, regression is a technique used for analyzing the relationships between variables; however, it has also been widely used for forecasting. In this study, six different regression models are used for our QoS time series forecasting problem: *simple linear* and *simple non-linear* regressions, *polynomial* and *non-linear polynomial* regressions, and *multiple linear* and *multiple non-linear* regressions.

6.2.1. Simple linear regression

In this model, the form (expression) of the regression model (i.e., the predictor) is as follows:

$$F_t = \alpha + \beta \cdot (t) \quad (16)$$

where α is an intercept and β is a coefficient of the second term; their values are assigned by a selected regression (training) approach such as the least-square-error method. In this model, the prediction input (expression/predictor input, i.e., t) is not a past observation (historical data) of the regressed (forecast) time series; instead, the past observations are used during the regression model's training process, and the prediction input is the index of the time point to be forecast.

6.2.2. Simple non-linear regression

In time series forecasting, a simple way to obtain a non-linear variation of a linear regression model is to use *natural logarithms* to transform the time series data and prediction input from items with linearity into items that are non-linear [46]. Below, we define the non-linear version of our simple linear regression model:

$$F_t = \exp(\alpha + \beta \cdot (\log(t))) \quad (17)$$

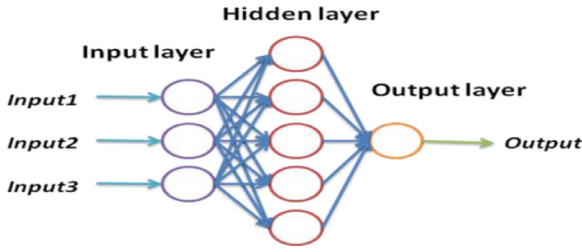


Fig. 7. The structure of an ANN.

where $\log()$ is a natural logarithm function, and $\exp()$ is a natural exponential function. Here, the prediction inputs must first be naturally logarized because the model was trained (regressed) to fit naturally logarized historical time series (QoS observations). Then, the model's output must be naturally exponentialized to acquire its final forecasting result (eliminating the effect of the natural logarithm).

6.2.3. Polynomial regression

The regression expression (predictor) in this model is a polynomial, and like the previous two regression models, it has only one prediction input (t). This regression model is defined as follows:

$$F_t = \alpha + \beta_1 \cdot (t) + \beta_2 \cdot (t^2) + \beta_3 \cdot (t^3) + \dots + \beta_i \cdot (t^i) \quad (18)$$

where α is an intercept, $\beta_1, \beta_2, \dots$ are coefficients; their values are assigned by the selected regression (training) approach. In this study, we automatically determine the degree (i.e., the number of i) of a polynomial by Akaike Information Criterion (AIC).

6.2.4. Non-linear polynomial regression

As noted previously, we use natural logarithms and natural exponents to obtain a non-linear version of the polynomial regression model:

$$F_t = \exp(\alpha + \beta_1 \cdot (\log(t)) + \beta_2 \cdot (\log(t)^2) + \beta_3 \cdot (\log(t)^3) + \dots + \beta_i \cdot (\log(t)^i)) \quad (19)$$

6.2.5. Multiple linear regression

The previous four regression models have only one predictor variable (t); however, a multiple regression model includes multiple predictor variables. Thus, in this study, except for the predictor variable t , we include seasonal variables in our multiple regression model, which is defined as follows:

$$F_t = \alpha + \beta_1 \cdot (t) + \beta_2 \cdot (s_2) + \beta_3 \cdot (s_2) + \dots + \beta_k \cdot (s_k) \quad (20)$$

where $s_2 \dots s_k$ are the seasonal variables, and k is the frequency of the regressed time series data. In time series forecasting, frequency is the number of observations per season, which is different from the definition and meaning used in physics [46]

6.2.6. Multiple non-linear regression

We use the same approach to obtain a non-linear version of multiple regression, which is defined below:

$$F_t = \exp(\alpha + \beta_1 \cdot (\log(t)) + \beta_2 \cdot (s_2) + \beta_3 \cdot (s_2) + \dots + \beta_j \cdot (s_k)) \quad (21)$$

6.3. Machine learning method

This section introduces artificial neural networks (ANNs) as the main machine learning method for QoS forecasting. An ANN (Fig. 7) consists of an input layer, zero or more hidden layers, and an output layer; each layer has a number of neurons connected to the neurons in the following layer. In ANNs, both the number of hidden layers and the number of neurons in each hidden layer are

variable and problem-specific. Except for the neurons in the input layer, each ANN neuron has an activation function that determines whether the output connections of that neuron are activated based on weighted input coming from the previous layer's neurons. In ANNs, each neuron-to-neuron connection has a weight with a value that is assigned during the ANN training process; thus, each neuron's received input value is a weighted combination of the values transmitted by the neuron's input connections. Finally, the output layer generates the final result of an ANN's internal calculation. Currently, there are several training methods for ANNs, among which *forward-back-propagation* may be the most common.

6.4. State-of-the-art methods

In this section, the two most widely used time series forecasting methods, i.e., *ARIMA models* and *exponential smoothing*, are introduced.

6.4.1. Auto-regressive integrated moving average models

ARIMA models for time series forecasting include three components, namely, the *Auto-Regressive* (AR), *Integrated* (I), and *Moving Average* (MA) parts. For each component, there is an order number (denoted by p for the AR part, d for the I part, and q for the MA part) that must be determined before an ARIMA model can be used to make forecasts; suitable values are assigned by analyzing the past data (training data) for the time series to be forecasted. Before the order numbers p and q can be determined, the order number d for the I part must be determined because it indicates how many times the differencing operation must be applied to the time series (training time series) to make the time series stationary in both its trend and variation (stationarity is one of the basic assumptions of ARIMA models). The order numbers p and q indicate that there are p and q predictor variables in the constructed ARIMA model's AR and MA parts, respectively. Similar to the regression models, each predictor variable also has a corresponding coefficient to adjust the effect of the predictor variable when forecasting. Thus, once the three order numbers have been determined, the coefficient of each predictor variable included in the AR and MA parts must be specified. The form of the ARIMA models is as follows:

$$F_{cur+1} = \sum_{i=0}^p \theta_i y_{cur-i} + \sum_{j=0}^q \theta_j \varepsilon_{cur-j} + \varepsilon_{cur+1} \quad (22)$$

where θ_i and θ_j represent the predictor variable coefficients, ε_{cur+1} is the current error, and ε_{cur-j} for $j=0, 1, \dots, j$ are the past errors. More detailed introductory and descriptive information on ARIMA models can be found in [25,32–35,46], including well-developed, sophisticated approaches that can be used to obtain a set of adequate order numbers and coefficients (i.e., an ARIMA model instance) for time series forecasting problems.

6.4.2. Exponential smoothing

Exponential smoothing actually represents a group of methods that include simple exponential smoothing (SES) and the representative Holt-Winters approach. The former is the simplest exponential smoothing method and can be considered a balance between the average method and the naïve method. Similar to the average method, the forecasting result of SES is a weighted summation of the past time series observations. However, in contrast to the average method, in which the weights are identical for all observations, or the naïve method, which gives full weight (100%) to the last observation, the SES weights exponentially decay as the observations get older. Thus, more recent observations have larger weights and contribute more to the forecasting result. A detailed explanation of SES can be found in Chapter 7.1 of [46]. The Holt-Winters method

is a representative method in time series forecasting. As noted in [25], ARIMA and Holt-Winters methods are widely used in time series modeling and forecasting. The Holt-Winters method has two variations: the additive model and the multiplicative model, each of which forecasts time series with different properties. An additive Holt-Winters model can be represented as follows:

$$F_{cur+1} = m_{cur} + b_{cur} + c_{cur} \quad (23)$$

where m_{cur} describes the level of the forecasted time series before time cur (training data), b_{cur} describes its trend, and c_{cur} describes its seasonality. The formulas used for obtaining the level (m_{cur}), trend (b_{cur}), and seasonality (c_{cur}) values of a forecasted time series can be found in Chapter 7.5 of [46] and Section 3.1.2 of [25], and a detailed introduction and description to the Holt-Winters method can be found in Chapter 7.5 of [46]. A list of exponential smoothing methods and a more detailed introduction can be found in Chapter 7 of [46].

6.5. Random search

In this study, we also include random search. Note that, in time series forecasting, random search is not a common and reliable method because its philosophy is to randomly select a predictor as its answer (solution) without any sophisticated procedure or sensible rules applied either during or before its search. In most time series forecasting cases, random search is not typically considered because it is unlikely to have an acceptable forecasting performance (accuracy). However, we include and implement random search for our evaluation of the SBSE validity of the proposed GP-based approach, which is discussed in detail in the next section. In this study, a random search is realized by configuring our GP to randomly select a chromosome (predictor) from the first generation of a GP evolution to be the result of the random search. The methods can also be categorized into *deterministic* and *indeterministic*. The latter methods include randomness; thus, given identical input and conditions, their results (predictors) would not necessarily be identical, in which case the predictor's forecasting result would also change. To adequately test and verify an indeterministic approach, the approach must be run repetitively, and the forecasting accuracies collected over multiple runs must be averaged to obtain the approach's true forecasting performance. Of the methods covered in this paper, our GP-based approach, ANNs, and random search are all indeterministic; thus, each must be executed repeatedly against the same QoS forecasting problem instance to know their true forecasting performance. This aspect of the methods is reviewed in our discussion of *Threats to Validity* presented in the Section 7.2.2.

7. Experiments and discussion

This section describes the performed experiments and provides a detailed discussion. In experiments section, we focus on the facts related to our experiments, including the experiment data, the hardware and programming environment, and the forecasting performance and comparisons. Subsequently, a discussion section comprises three independent parts: observations and discussion on the experiment results presented in the first subsection, a discussion on validity threats to our GP-based approach, and an evaluation of the SBSE validity of our approach.

7.1. Experiments

7.1.1. The QoS data

In this paper, a randomly selected real-world response-time series contained in the QoS dataset provided by [32] and freely available on the Web was used as our experimental data and case

Table 3

The common parameters, the GP parameters, and their values as set in our experiments.

Common parameter	Value
The number of TSs	20
The length of a TS	600
The size of available training resources (tl) in a TS	500
The size of testing data (pl) in a TS	100
GP parameter	Value
The number of repetitions for a TS	30
Fitness function	MAE
SelectionOperator	Binary tournament
PopulationsSize	2000
Number of generations	150
Crossover probability	0.9
Mutation probability	0.06
Function set	{+, −, ×, ÷, mod(), min(), max(), power(), sine(), cosine(), tangent(), absolute(), floor(), ceil(), round(), log(), exponential()}
Terminal set	{ Y_{t-1} , Y_{t-2} , ..., Y_{t-15} , random constants between −10 and 10}

study. The dataset was created by invoking ten real-world WSs and recording their QoS values hourly for four months; a detailed description can be found in [32]. This dataset has also been used as test data by other QoS forecasting studies such as [32,33]; thus, it should be reliable in dynamic QoS forecasting research. Because the length of the time series is approximately 3000 (the response time of the same real-world WS recorded at approximately 3000 consecutive time points), which is too long for a real-world QoS forecasting scenario, we split the time series into *timeseries segments* (TSs), allowing us to run our experiments multiple times with different data. In this paper, the proposed approach and the method introduced here are used to forecast the first TS for the first experiment (Experiment 1), the second TS for the second experiment (Experiment 2), and so on.

7.1.2. The hardware and programming environments

The proposed GP-based approach and the compared methods are all implemented and run on Apple Mac with OS X 10.8.5 operating system, 1.3 GHz Intel Core i5 processor, and 4GB DDR3 RAM memory. Regarding the programming environments, our proposed GP-based approach is implemented in Java using a Java-based GAs/GP framework called the *Java Genetic Algorithms and Genetic Programming Package* (JGAP) [52]. The compared methods are implemented in the R language using a useful and reliable R time series forecasting package called *forecast* [46,53]. The compared methods use R functions provided by the *forecast* package, including *meanf* (the average method), *naïve* (the naïve method), *msts* (the seasonal naïve method), *rwf* (the drift method), *tslm* (the regression methods), *nnetar* (the ANNs), *auto.arima* (the ARIMA models), and *ets* (the exponential smoothing methods).

7.1.3. Experiment configuration

This section presents the parameters used in our experiments and their values. The parameters can be divided into two parts, namely, the *common parameters* and *those that are needed only in GP* (see Table 3). The parameters in the first part are required by and common to all the reviewed QoS forecasting solutions. The common parameters define the QoS forecasting problem described for this study. In our experiments, the extended time series data introduced in Section 7.1.1 are split into 20 time series segments (TSs), creating 20 different experiments. Note that there are partial overlappings between the ranges of these TSs, although they are different TSs. The length of a TS is 600 in this study. Each TS

Table 4

Our experimental (testing) results shown as MAEs (units are milliseconds).

	TS1	TS2	TS3	TS4	TS5	TS6	TS7	TS8	TS9	TS10	TS11	TS12	TS13	TS14	TS15	TS16	TS17	TS18	TS19	TS20
GP	1374	888	766	2729	4459	781	1013	1376	1934	968	1196	1088	1123	1300	1073	1031	1075	2743	1368	990
Ave.	4856	4114	4002	6214	8319	3197	3161	3263	3248	2228	1412	1286	1450	1465	1206	1235	1053	2676	1195	1013
Nai.	2041	1261	954	2876	4991	1074	1355	1964	2475	1110	1485	1310	1880	2040	1579	1534	1461	2486	1748	1319
DSN	2389	1196	1315	4608	6679	1213	1335	2079	2107	1716	1783	1297	1468	2144	1256	1504	1494	3129	3198	1467
WSN	16,984	2105	872	6111	9566	10,527	6270	1601	2523	1615	2204	1778	1256	1304	1803	1409	1636	3015	1642	3021
Dri.	2219	1913	1521	6803	6364	743	921	8474	4853	1051	1440	1134	1172	3174	1117	971	938	2885	1116	1127
SLR	10,191	3944	1251	12,473	7178	10,084	5748	2165	3737	2806	1870	1373	1209	1227	1216	1152	1000	2607	1653	1647
SNR	3445	2109	1241	6533	8632	3884	3242	2456	1883	1149	1343	1218	1237	1234	1087	1031	941	2598	1111	951
PR	43,664	31,198	16,670	36,562	28,400	28,213	7023	34,559	20,463	6401	1231	1493	1089	2887	4096	4149	1282	2909	19,062	6218
NPR	2668	1814	2134	5665	7261	12,318	1262	2987	2217	2990	1523	981	1080	1243	1140	1030	943	2602	1440	1375
MLR	11,588	6598	839	16,612	7149	9722	7074	1858	3271	3641	1891	1356	2340	1236	1016	970	1139	2881	1363	1630
MNR	4084	2873	1389	7366	8641	3480	3915	2066	2173	1235	1402	1283	1481	1287	1024	986	1135	2849	1148	1069
ANNs	2902	1368	1569	5686	4949	862	1236	2194	2165	1208	1304	1209	1388	1605	1224	1219	1059	2679	1263	951
ARIMA	1843	1027	826	2599	4231	908	1118	1603	2120	1087	1447	1250	1473	1505	1225	1239	1058	2672	1451	1164
ES	1767	905	823	2380	4342	899	1173	1580	2111	1237	1504	1128	1267	1601	1147	1213	1053	2698	1545	1085

is divided into a *first part*, which is used for training/learning (by each forecasting solution in the first phase of the defined forecasting problem to find a predictor), and a *second part*, which is used for testing (in the third phase to measure the forecasting accuracy of the predictor found). In this study, the training and testing lengths are 500 (corresponding to tl defined in the last paragraph of Section 4.1) and 100 (corresponding to pl defined in the same paragraph as tl), respectively. Considering possible real-world QoS forecasting scenarios, the test data length (pl) was kept short because it is both unlikely and unnecessary to forecast all future dynamic QoS values using the same predictor. In time series forecasting, more accurate forecasting can be achieved by using predictors that have been generated and trained with more recent data because of varying data generation processes [21]. Thus, in terms of forecasting accuracy, it would be better to replace an old predictor with a new predictor (in our case, after 100 future time points) trained using more recent data (the newly received QoS values over those 100 time points). However, an excessively small pl value increases the cost of forecasting (the number of iterations required to generate/train/search a predictor) and may have no or only a small enhancing effect on the forecasting accuracy. For example, the most extreme case would be to create a new predictor every time to predict the next one-step-ahead QoS value (i.e., let pl be 1). An optimal pl value that takes both cost and forecasting accuracy into account needs to be further investigated, although this is beyond the scope of this initial study. Regarding the size of the available training/learning resources (tl), more training/learning data would seem to intuitively lead to more accurate forecasting; however, this is not always the case in many time series forecasting studies. Larger training/learning resources are usually accompanied by higher training/learning/searching costs (for example, in GP, using more training data requires more time to calculate the fitness value of a chromosome). Thus, the most suitable tl value for QoS time series forecasting must also be further investigated, although this is not the main focus of this paper. Although optimal tl and pl values can only be found by further investigation, we believe that the settings used here are reasonable in the real world and adequate for an initial study. In the current experiments, the most important thing is to ensure that the compared forecasting methods and our proposed approach are run impartially and evaluated under the same conditions.

The GP parameters are dedicated solely to our GP-based approach. In this study, we set the value of these parameters based on common knowledge and experience in GP. We also refer to other GP-based time series forecasting studies, such as [49–51], for insight on GP parameter settings. For example, we let our crossover probability be much larger than the mutation probability because

this is a common convention in most GA/GP-based approaches. We also perform a set of GP runs (repetitions) for the same TS because, as noted in [21], a GP application's results may vary from run to run (indeterministic approach); therefore, it is common practice to run a GP multiple times. In this study, our experiments simply use the MAE defined in Section 4.2 as the GP's fitness function because it is how we quantitatively measure the forecasting accuracy; thus, it is both intuitive and reasonable to use the MAE to score the chromosomes (predictors) that appear during GP evolution process. Again, similar to the common parameters, the optimal value for each GP parameter such as population size, number of generations, crossover and mutation probabilities, and their best combination can be known only by further investigations and trials, which are beyond the scope of this paper. Regarding the function set and terminal set of GP, in theory, unsuitable function and terminal elements (which create worse predictors) would naturally be eliminated during the GP evolution process; thus, we choose the elements contained in the two sets loosely and diversely, expecting that any elements unsuitable and useless for accurate forecasting (precise predictors) would be automatically and naturally ruled out. In addition, richer function and terminal sets also enlarge the search space covered by GP, which increases the possibility of obtaining accurate predictors.

7.1.4. Forecasting performance and comparison

This section describes our numerical experiment results in detail. The experiments are simulations of the defined QoS forecasting problem (Section 4.1), and the values presented below are the results of the third problem phase (assessing the forecasting accuracy of each predictor with unseen testing time series data). First, the experiment results measured using the MAE and MAPE defined in Section 4.2 are depicted in Tables 4 and 5, respectively. In both tables, the columns to the right of the first column represent the different time series segments. The proposed GP-based approach and the compared methods are designated by abbreviations in the table rows (in the first column). In both tables, GP represents the proposed GP-based forecasting approach. The abbreviations Ave., Nai., DSN, WSN, and Dri. are the average, naïve, daily seasonal naïve, weekly seasonal naïve, and drift methods introduced in Section 6.1, respectively. The abbreviations SLR, SNR, PR, NPR, MLR, and MNR indicate the simple linear regression, simple non-linear regression, polynomial regression, non-linear polynomial regression, multiple linear regression, and multiple non-linear regression presented in Section 6.2, respectively. The abbreviation ANNs (artificial neural networks) represents the only machine learning-based forecasting method included and compared in this study. Intuitively, the ARIMA models and exponential smoothing, both introduced in Section 6.4, are represented by the symbols

Table 5

Our experimental (testing) results shown as MAPEs (units are percentages).

	TS1	TS2	TS3	TS4	TS5	TS6	TS7	TS8	TS9	TS10	TS11	TS12	TS13	TS14	TS15	TS16	TS17	TS18	TS19	TS20
GP	15.4	12.6	12.4	19.4	34.6	13	14.8	17.3	21.8	14.9	15.5	18	13.5	18.8	17.3	17.6	17.3	20.7	20.5	15.6
Ave.	85.5	76.9	77.1	62.6	51.5	64.3	62.5	56.7	51.9	42.3	20.4	24	21.6	24.2	21.6	23.3	18.2	21.1	18.3	17.3
Nai	28.5	19.6	16	21	36.5	19.1	21.5	28.6	30.6	18.9	19.9	23.4	28.6	33.1	28.1	27.8	24.8	35.7	27.6	22.6
DSN	34.1	17.8	23.1	30.2	73.5	21.7	21.7	32.7	25.5	28.8	28.5	22.5	19.9	36.1	22.4	27.5	25.9	29.6	58.3	24.8
WSN	318.9	36.1	15.1	37.7	52.8	206.5	125.4	20.2	31.4	27.1	33.8	32.4	16.9	20.6	33.5	26.4	27.8	25.8	24.8	60.7
Dri.	33.5	33.8	28.4	40.8	70.9	12.5	13	155.5	84.1	15.1	17.9	20.3	13.7	62.4	17	16.2	15.2	21.7	15.4	16.8
SLR	187.6	73.6	20.2	106.7	52.8	198.5	114.4	34.5	48.5	45.6	30.1	26.2	16.1	18	21.9	21.3	16.9	18.9	28.9	30.8
SNR	57.8	37.8	22.7	38	49.7	78.3	64.1	40.5	21.7	16.6	18.9	22.4	16.8	18.2	18.6	18.2	15.3	18.6	15.5	15.8
PR	809.5	549.2	316	314.3	354.5	544.6	125.5	620.3	322.9	119.7	15.5	23.6	13.1	55.9	69.9	86.8	19.7	27.5	332.6	105.6
NPR	42.5	27	36.4	44.7	63.7	241.8	20.6	44.2	23.5	57.6	22.8	15.9	12.8	18.4	20	18.2	14.4	17.9	24.3	25.2
MLR	214.3	123.6	13.6	159.5	52.9	191.5	140.4	28.3	40.6	60.6	30.6	25.8	40.7	18.3	15.6	15.9	17.2	21.6	22.5	30.5
MNR	70.4	52.9	25.7	47.3	49.7	70.2	77.9	32.5	28.5	18	20.2	24	22.4	19.7	15.7	15.9	17.1	21.1	15.4	18.6
ANNs	43.3	21.9	28.6	39.6	41.7	15.5	20.2	36.6	27.2	20.4	18.5	22.6	20	26.4	22	23	18.4	20.8	17.8	15.7
ARIMA	25.3	15.6	13.9	18.7	36.4	16	17.7	21.9	28	18.4	21.8	23.3	21.9	24.7	22	23.4	18.4	20.9	22.5	20
ES	24	13.4	13.9	17.5	37.6	15.8	18.6	22	27.7	21.1	22.6	19.9	17.1	27	19.9	22.7	18.2	21.8	25.1	18.3

Table 6

Our comparison results shown as RPAIs (units are percentages).

	TS1	TS2	TS3	TS4	TS5	TS6	TS7	TS8	TS9	TS10	TS11	TS12	TS13	TS14	TS15	TS16	TS17	TS18	TS19	TS20
Ave.	82	83.6	83.9	69	32.8	79.8	76.3	69.5	58	64.8	24	25	37.5	22.3	19.9	24.5	4.9	1.9	−12	9.8
Nai	46	35.7	22.5	7.6	5.2	31.9	31.2	39.5	28.8	21.2	22.1	23.1	52.8	43.2	38.4	36.7	30.2	42	25.7	31
DSN	54.8	29.2	46.3	35.8	52.9	40.1	31.8	47.1	14.5	48.3	45.6	20	32.2	47.9	22.8	36	33.2	30.1	64.8	37.1
WSN	95.2	65.1	17.9	48.5	34.5	93.7	88.2	14.4	30.6	45	54.1	44.4	20.1	8.7	48.4	33.3	37.8	19.8	17.3	74.3
Dri.	54	62.7	56.3	52.5	51.2	−4	−13.8	88.9	74.1	1.3	13.4	11.3	1.5	69.9	−1.8	−8.6	−13.8	4.6	−33.1	7.1
SLR	91.8	82.9	38.6	81.8	34.5	93.5	87.1	49.9	55.1	67.3	48.5	31.3	16.1	−4.4	21	17.4	−2.4	−9.5	29.1	49.4
SNR	73.4	66.7	45.4	48.9	30.4	83.4	76.9	57.3	−0.5	10.2	18	19.6	19.6	−3.3	7	3.3	−13.1	−11.3	−32.3	1.3
PR	98.1	97.7	96.1	93.8	90.2	97.6	88.2	97.2	93.2	87.6	0	23.7	−3.1	66.4	75.3	79.7	12.2	24.7	93.8	85.2
NPR	63.8	53.3	65.9	56.6	45.7	94.6	28.2	60.9	7.2	74.1	32	−13.2	−5.5	−2.2	13.5	3.3	−20.1	−15.6	15.6	38.1
MLR	92.8	89.8	8.8	87.8	34.6	93.2	89.5	38.9	46.3	75.4	49.3	30.2	66.8	−2.7	−10.9	−10.7	−0.6	4.2	8.9	48.9
MNR	78.1	76.2	51.8	59	30.4	81.5	81	46.8	23.5	17.2	23.3	25	39.7	4.6	−10.2	−10.7	−1.2	1.9	−33.1	16.1
ANNs	64.4	42.5	56.6	51	17	16.1	26.7	52.7	19.9	27	16.2	20.4	32.5	28.8	21.4	23.5	6	0.5	−15.2	0.6
ARIMA	39.1	19.2	10.8	−3.7	4.9	18.8	16.4	21	22.1	19	28.9	22.7	38.4	23.9	21.4	24.8	6	1	8.9	22
ES	35.8	6	10.8	−10.9	8	17.7	20.4	21.4	21.3	29.4	31.4	9.5	21.1	30.4	13.1	22.5	4.9	5	18.3	14.8

ARIMA and ES, respectively. Finally, we do not show the random search (Section 6.5) results because, as expected, its forecasting accuracy is much worse than either our proposed approach or the other compared methods. In the following tables, for each experiment (TS), we indicate the best forecasting method among the 14 compared methods by showing its value in blue except when the proposed approach (GP) is the best, in which case the value appears in red.

To include intuitive numerical results regarding the comparisons between our GP-based approach and each of the compared methods, we show our comparisons via their RPAIs in Table 6, where we use the MAPE of each compared method as $RMAE_{p1}$ and the MAPE of our approach as $RMAE_{p2}$ (all of the original MAPEs are presented in Table 5). In this table, negative values are red, meaning that the proposed GP approach performed worse than the corresponding compared method in forecasting accuracy measured using the MAPE for that experiment (TS).

To provide an overview of the forecasting accuracy of each of the tested forecasting solutions, Table 7 shows calculated average, minimum, maximum, and standard deviations of the MAEs, MAPEs, and RPAIs for all of the experiments (TS1–TS20).

Finally, we demonstrate one of the predictors that evolved through GP in our experiments for the reader to review, where P_t represents the QoS value to be forecasted:

$$P_t = \max((\min((\text{ceil}(\text{ceil}(\max((\min(Yt-1, (\text{floor}(Yt-2))), (\min((\max(Yt-6, (\min((\text{round}(\max(Yt-5, (\text{ceil}(\text{floor}(Yt-2))), ((\min((\text{round}(\min((\max(Yt-1, Yt-15))), (\min(Yt-7, Yt-9))), Yt-15)) - 7.350308))), (\max((\max((\min(Yt-7, Yt-9)), (\text{floor}(Yt-3 - 6.827459))), (\text{round}(Yt-1))))))), (\text{round}(\max(Yt-4, (\min((\text{round}((\max(Yt-1, Yt-6)) - (\min((\min(Yt-1, \text{ceil}(\text{floor}(Yt-2))), (\text{round}(\min((\text{floor}(Yt-3 - 6.827459))), Yt-7))))))), Yt-7))))))), (\text{floor}(\max(Yt-5, (\text{floor}(\text{round}(\text{floor}(Yt-2))))))), (\min((\text{round}(Yt-3), (\max((\min(Yt-3, ((\min((\max(Yt-1, Yt-15))), (\max(Yt-5, (\max((\min(Yt-3, (\max(Yt-5, (\min((\text{ceil}(\text{floor}(Yt-2))), (\max(Yt-5, (\max(Yt-5, (\min(Yt-1, (\max(-2.407012, Yt-15))))))))))))), (\min(Yt-4, Yt-13)))))) - 0.10299921))), (\min((\max(Yt-5, (\text{ceil}(\min((\text{floor}(Yt-2), (\text{round}(\min((\text{floor}(Yt-3 - 6.827459))), Yt-7))))))), Yt-7))))))), Yt-7))))))$$

(floor Yt-2))), (round (min((floor (Yt-3 - 6.827459)), Yt-7))))), (floor (max(Yt-5, (floor (round (floor Yt-2))))))), (min((round Yt-3), (max((min(Yt-3, ((min((max(Yt-1, Yt-15))), (max(Yt-5, (max((min(Yt-3, (max(Yt-5, (min((ceil (floor Yt-2))), (max(Yt-5, (max(Yt-5, (min(Yt-1, (max(-2.407012, Yt-15))))))))))))), (min(Yt-4, Yt-13)))))) - 0.10299921))), (min((max(Yt-5, (ceil (min((floor Yt-2), (round (min((floor Yt-3 - 6.827459))), Yt-7))))))), Yt-7))))))

This example of a GP-based predictor is included solely so the reader can see what a GP-evolved predictor looks like. It is obvious that the structure and form of the predictor presented above is more complex than those of the predictors generated by the methods introduced in Section 6; thus, it can be said that this predictor is more sophisticated and complex than the conventional time series predictors listed in Section 6. A complicated predictor is more likely to capture and better model the patterns and correlations that implicitly exist in the fitted time series and, thus, to produce more accurate predictions, as shown in our experimental results and comparisons. No conventional time series method could possibly generate such a complex predictor. The next section discusses the advantages of the predictors generated by using the proposed GP-based approach (their forecasting accuracy and predictor properties such as white-box-based and non-linearity).

7.2. Discussion

7.2.1. Observations and discussion of the experiment results

The discussion in this section comprises two different parts. The first part concerns the numerical and empirical experimental results presented in Section 7.1.4. Moreover, the differences (comparison) in the forecasting solutions (a comparison of their charac-

Table 7

A statistical overview of the experimental results based on the MAEs, MAPEs, and RPAIs.

	Ave. MAE	Min. MAE	Max. MAE	S.D. MAE	Ave. MAPE	Min. MAPE	Max. MAPE	S.D. MAPE	Ave. RPAI	Min. RPAI	Max. RPAI	S.D. RPAI
GP	1463	766	4459	896	17.5	12.4	34.6	4.8	–	–	–	–
Ave.	2829	1013	8319	1949	42	17.3	85.5	23.5	42.9	–12	83.9	31.4
Nai	1847	954	4991	899	25.5	16	36.5	5.8	30.7	5.2	52.8	11.9
DSN	2168	1196	6679	1365	30.2	17.8	73.5	13.3	38.5	14.5	64.8	12.5
WSN	3862	872	16,984	4142	58.6	15.1	318.9	75.9	44.6	8.7	95.2	26.8
Dri.	2496	743	8474	2289	35.2	12.5	155.5	35	23.7	–33.1	88.9	35.8
SLR	3726	1000	12,473	3524	55.5	16.1	198.5	54.8	43.9	–9.5	93.5	32.6
SNR	2366	941	8632	2025	30.2	15.3	78.3	18.7	25.1	–32.3	83.4	33.6
PR	14,878	1089	43,664	14,202	241.3	13.1	809.5	237.4	69.9	–3.1	98.1	36
NPR	2733	943	12,318	2775	39.5	12.8	241.8	49.7	29.8	–20.1	94.6	33.7
MLR	4208	839	16,612	4302	63.2	13.6	214.3	64.2	42	–10.9	93.2	37.6
MNR	2544	986	8641	2122	33.1	15.4	77.9	20.5	30	–33.1	81.5	33.7
ANNs	1902	862	5686	1295	25	15.5	43.3	8.6	25.4	–15.2	64.4	20.3
ARIMA	1592	826	4231	802	21.5	13.9	36.4	4.9	18.3	–3.7	39.1	11.1
ES	1572	823	4342	819	21.2	13.4	37.6	5.4	16.5	–10.9	35.8	11.2

teristics) is discussed and described in the second part (their pros and cons in terms of their principles rather than their numerical forecasting performance/accuracy).

We begin our discussion with the outcomes presented in Tables 4 and 5. Initially, it is difficult to immediately determine which tested forecasting solution is best because no single solution dominates (with the lowest MAE and MAPE) over all of the experiments (TS1–TS20). However, our approach has the lowest forecasting error in 6 MAE-measured experiments (Table 4) and in 8 MAPE-measured experiments (Table 5), which is more than any of the compared solution. In terms of the number of experiments (TSs) where one model dominates, the second best solution is simple non-linear regression (SNR) and the drift method (*Dri.*) in the MAE-measured experiments and non-linear polynomial regression in the MAPE-measured experiments. In other words, using our approach, these models achieve the best forecasting accuracy (lowest forecasting error). In a real-world QoS forecasting context, it is impossible to switch between different forecasting solutions for different TSs of the same service because, even with the assistance of a human expert, it is difficult to anticipate which forecasting solution would be the most suitable for the next incoming as-yet-unseen TS. In fact, in real-world situations, performing QoS forecasting with the assistance of human experts is both unrealistic and infeasible because of the numerous services that currently exist on the Web. Thus, a more reasonable and feasible method is to make forecasts for the same service's different TSs using the same (automated) forecasting approach/method. Under such circumstances, our GP-based approach would be a sensible choice, as discussed below (concerning the data in Table 7), because our approach has the lowest average forecasting error (as measured by the MAE and MAPE) among the tested forecasting solutions.

According to Table 6, our GP-based approach is better than the compared methods (with positive RPAIs) in 89% of the comparisons, and the average MAPE improvement percentage (the average of the positive RPAIs) is approximately 40%. In the other 11% of the comparisons, our approach is worse in forecasting accuracy based on the MAPEs (with negative RPAIs) by an average of -10.6%. Using these quantitative comparison results, our approach outperforms the compared methods in most of the tested TSs (89% of the cases), and this performance improvement is significant (by 40% on average). However, our proposed approach was defeated in only 11% of the comparisons, and the average extent of that defeat is insignificant (only -10.6%).

In Table 7, it is evident that our approach has the best average forecasting performance based on both the MAE and MAPE; thus, as discussed earlier, it is also most likely to have the best overall forecasting accuracy. In Table 7, the standard deviation presents

the degree of variation in the forecasting accuracy (i.e., forecasting stability). A forecasting approach with a large standard deviation has significant variations in forecasting errors, which means that it may not be a stable forecasting solution (sometimes its forecasting errors are very large, while others are quite small, and the errors deviate widely from their average). Although our approach is not the solution with the lowest standard deviation in the MAE-measured experiments, it is the one with the lowest average MAE-measured forecasting accuracy. An approach with a small standard deviation but that is not ideal in terms of forecasting performance measured by either the MAE or MAPE may imply that the approach always has the same or similar types of forecasting errors. In the MAPE-measured experiments, the proposed approach has the best (lowest) MAPE and standard deviation among the tested forecasting solutions. Regarding the MAE-measured experiments, our approach has the lowest and third lowest MAE and standard deviation, respectively. The third lowest standard deviation (896) is only slightly higher than the lowest standard deviation (802). Finally, the average RPAIs show that our approach is better than any of the compared methods on average (all have positive average RPAIs) and provide the numerical extent (the percentage) to which GP outperforms the others.

In our experimental results, the two most powerful competitors of our GP-based approach are ARIMA models and exponential smoothing. This is not surprising because, as noted in [46], they are the two most commonly used and well-developed methods in time series forecasting. This result also supports the decisions made by the authors of the studies reviewed and discussed in Section 3.1, of which more than half based their forecasting approaches on ARIMA models. Our empirical experimental results agree with their forecasting model selections in terms of the forecasting accuracy measured according to the MAE and MAPE; although ARIMA models perform worse than our proposed approach, they are indeed better than any of the other compared methods. Furthermore, regarding the forecasting accuracy, exponential smoothing can perform even better than ARIMA models over many TSs (experiments; on average), although we have yet to observe any approaches that apply exponential smoothing for QoS forecasting. We believe that it is both promising and worthwhile to develop a QoS forecasting approach based on exponential smoothing because it is the second best method in our empirical experiment results—even better than the ARIMA models commonly used in current research. Our empirical experiment results may also show that in studies applying ANNs, such as [19,24], the models may not be as effective and efficient as their authors originally expected. In [19], the authors adopted ANNs because of their flexibility and capacity and they are the biggest challengers to conventional time series forecasting

methods. In our experiments, in terms of the average forecasting accuracy, while it is true that ANNs are better than many of the other conventional time series methods compared herein, they are still worse than our proposed approach and worse than the two most commonly adopted time series forecasting methods, namely, ARIMA models and exponential smoothing. Thus, these results do not agree with the expectations of such ANN-based studies. Finally, it is surprising that the naïve method is as good as ANNs in our performed experiments because the predictor structure and forecasting philosophy of ANNs are much more complicated than those of the naïve method. In situations where time and cost restrictions preclude constructing a QoS predictor, the naïve method can be used to quickly obtain a usable predictor.

The above discussion and observations are all consequences of the execution (forecasting) results analyzed to determine the most suitable methods and approaches. In contrast, we focus below on the principles (the nature and features of each forecasting solution). The following analysis and discussion is presented and organized with the assistance of several classifications for the forecasting method/approaches. In addition to the *mathematical expression-based vs. non-mathematical expression-based* classification introduced in the beginning of Section 6, there are several other ways to classify the compared time series forecasting solutions, for example, (1) *black-box-based vs. white-box-based*, (2) *linear vs. non-linear*, and (3) *classical vs. modern heuristic* [21]. A black-box-based forecasting approach means that the method used to obtain forecasting result was produced by a predictor is not understandable or observable; the predictor is similar to a black box. However, using a white-box-based forecasting approach, it is possible to observe the generation process of a forecasting result (how a forecasting result was produced) because the predictors found by the approach can be observed and understood by humans; thus, these are called white-box approaches. In the forecasting solutions included here, only ANNs are black-box approaches because, as discussed and illustrated in Section 6.3, each ANN is intricate; its output (forecasting results) is calculated and determined by a complicated interplay between its internal neurons, which is difficult for human beings to either observe or understand. In contrast, for white-box-based solutions, the generated predictors are both understandable and readable by humans, which is helpful for forecasting. Furthermore, in this paper, all of the included white-box-based solutions are mathematical expression-based approaches, meaning that the predictors generated and found by these approaches are equations that can easily be read and understood by humans and are also helpful in analyzing and understanding the concept being forecast (in this paper, the dynamic QoS of a WS). For example, if the most suitable forecasting model for the response time of a WS is found and determined to be an AR model with an order of three, we can reasonably make an inference that the real value of the response time of the WS is strongly related to (determined by) its previous three observations, and the relationship between them can be explained simply by a weighted summation. In addition, in the coefficients of the AR model, the weight that each previous observation occupies in a forecasting result (its contribution to the forecasting result) can be observed and understood easily. With mathematical expression-based predictors (forecasting approaches), the relationship between the set of predictor variables that is used to construct a predictor (e.g., the past observations) is explained and encoded by several functions (e.g., mathematical operators) that are comprehensible to and explainable by humans. This same information cannot be obtained when using ANNs, which are non-mathematical expression-based forecasting solutions due to their internal intricacy, as discussed earlier.

As noted in [34], the majority of real-world QoS time series for WSs are non-linear; therefore, models that forecast QoS time se-

ries using linear forecasting solutions, such as the benchmark approaches introduced in Section 6.1, the linear regression models presented in Section 6.2, and the ARIMA models defined in Section 6.4, are neither reasonable or efficient. In [34], the authors proved that modeling and forecasting QoS time series based on the linearity of the received training QoS data is better than using only linear ARIMA models. Their approach selects a linear forecasting model when the incoming training data are linear; otherwise, it selects a non-linear forecasting method. Similarly, GP is able to automatically evolve adequate linear or non-linear expressions (forecasting models) based on the training acquired from existing QoS data because its search space covers all possible combinations (expressions) of the elements specified in the function and terminal sets. To fit a linear QoS time series, GP would evolve a linear expression; otherwise, it would evolve (the found predictor) a non-linear expression. Furthermore, one of the main differences between the included classical time series methods and the modern heuristic approaches is linearity. Most of the classical methods, such as the benchmark approaches and the ARIMA models, are linear models. Although we have introduced a non-linear version for each included linear regression model (classical method), that non-linearity is in fact obtained by a data transformation (from non-linear data to linear time series that can be handled (modeled) by a linear regression model); thus, the nature of these non-linear regression models is still linear (expression structure). They are not similar to the modern heuristic approaches (ANNs and GP); their principles and predictor forms are non-linear. Therefore, problems may occur when the linearity transformation does not function well. An important advantage of the modern heuristic forecasting approaches is that there is no need to pre-describe the form of the predictors (in classical methods, the predictor form/structure is specified in advance, as presented in Section 6). GP allows the most fit predictor (form) to be automatically discovered during the evolution process [21]. A more detailed discussion of the advantages of using the modern heuristic approaches can be found in [20,23].

From the previous discussion, GP lies on the better side of these classifications, meaning that it uses both white-box-based and expression-based predictors, and its heuristic search covers both the linear and non-linear expression space.

7.2.2. The threats to validity

In this section, we discuss the validity threats of our GP-based approach. First, a serious threat to our approach's practical usability is that GP requires time to evolve because it is an evolutionary paradigm involving repetitive trials and the use of a fitness function to score candidate solutions. In our experiments, the time required by GP to find a predictor is much longer than the compared methods. However, as noted in Section 3.1.4 of [23], “*quality overrides speed*”; although GP takes more time than the other methods, its overall forecasting quality is better and more stable than the other methods, as demonstrated in Section 7.1.4 and discussed in the first part of Section 7.2.1. Thus, although GP requires more time to determine a predictor, using GP is advantageous because its forecasting quality is better according to our results. In terms of the problem defined in Section 4, although GP takes considerably more time, the time available for it to solve a QoS forecasting problem (discovering an accurate predictor) is sufficient. For example, in our experiment configuration, we assume pl to be 100, which means that the current predictor will be used for forecasting the next 100 time points. Thus, the predictor has 100 h before a new predictor is due to replace the old predictor. According to our experiments, 100 h is more than sufficient for our GP-based approach to run once and evolve a predictor. In fact, even in an extreme forecasting configuration (small pl value), sufficient time is available for GP to run because GP requires less than one hour

to run once (in our experiments). However, the time required for GP to run could be further reduced by enhancing the hardware or adopting parallel computing techniques.

Next, we list and discuss the validity threats proposed by the authors of [54]; however, due to space limitations, we cover only the first two validity threat classes defined in [54], namely, the four conclusion and four internal validity threats.

7.2.2.1. Not accounting for random variation. A conclusion validity threat to GP and most evolutionary approaches is the initialization of the first-generation chromosomes, which is a random process in our GP and in most other evolutionary approaches [54]. In evolutionary approaches, the evolution and its results may preserve and be affected by the benefits of favorable first-generation chromosomes or the prejudices of poorly selected starting chromosomes [54]. Thus, to address this issue, as noted in Section 7.1.3, we run multiple GP evolutions (20 runs in this paper) for each QoS forecasting problem (the same TS). Then, we assess the GP forecasting performance (accuracy) of each TS by averaging the results of the multiple runs to avoid problems induced by the initial chromosomal benefits and prejudices (as suggested by the authors of [54]).

7.2.2.2. Lack of good descriptive statistics. As shown in Section 7.1.4 and discussed in the previous section, our GP experimental results are the average of multiple runs for the same QoS forecasting problem instance (TS), which meets the minimum requirement suggested in [54] that *papers should show the average of the observed results*. Moreover, we also present the min-max range and standard deviation of our GP experiment results, which was also suggested by the authors of [54].

7.2.2.3. Lack of a meaningful comparison baseline. As introduced in Section 6 and presented in Section 7.1.4, we compare more than ten different time series forecasting methods to our proposed GP-based approach, including random search, which is frequently used as a baseline comparison in SBSE research [54]. In our empirical experiments (comparisons), the compared methods include most of the representative time series forecasting methods; thus, the analyses, observations, and conclusions made based on the experiments and comparison results should be reliable, as requested by the authors of [54].

7.2.2.4. Lack of formal hypothesis and statistical tests. A comparison (such as the ones discussed in the previous section) *must be based on a formal hypothesis and must be evaluated by a proper statistical test* [54]. In this study, we formally define and use the RPAI to quantify our comparison results. In addition, we performed a set of formal statistical tests based on the *Diebold-Mariano test*, which compares the predictive accuracy of two compared time series methods and is provided in the R time series package *forecast* [46,53] introduced earlier. Using this test (an R function provided in the *forecast* package), we iteratively compared each of the compared time series methods with our GP-based approach. In each test, the null hypothesis is that the compared method is better than our GP-based approach in forecasting accuracy, and the alternative hypothesis is that the proposed GP-based approach is more accurate than the compared method. The testing results are consistent with the RPAI comparisons presented in Table 6. For example, in the 83 cases in which the null hypothesis was accepted (the compared method is more accurate), the corresponding RPAI values were larger than 20% (which means that the GP-based approach improves the forecasting accuracy of the compared method by more than 20%, and thus, is more accurate) only in 13 cases (16%), while most of the remaining 70 cases have negative RPAI values. In the 33 cases in which the RPAI values are negative (the compared method is better), the null hypothesis was rejected only in two cases.

7.2.2.5. Poor parameter settings. This is a type of internal validity threat included in [54]. The threat is concerned with the chance that the selected parameter values for a proposed solution and its compared approaches are not explicitly presented in the experimental design, which causes experiments to be non-reproducible. However, our experimental configuration values (parameter settings) are explicitly and completely described and discussed in Section 7.1.3; thus, this threat should not be a potential issue in our study.

7.2.2.6. Lack of discussion on code instrumentation. This threat is valid when the computer code used in experiments contains instrumentation and/or tweaks that favor certain (problem) instances or algorithms (approaches/methods) over others, thus influencing the observed results [54]. To control this threat, we plan to make the code used in our experiments publicly available on the Web, as suggested by the authors of [54]. With the specific and accessible experiment configuration clearly described in this paper and the provided code, other researchers can reproduce and test the experiments.

7.2.2.7. Lack of clarity concerning data collection tools and procedures. As noted previously, our experiments used a real-world QoS time series dataset created by the authors of [32] that is publicly available on the Web. Their original paper clearly described how the real-world QoS values were collected and recorded (including the data collection tools and procedures used) along with a detailed specification of the dataset; therefore, this issue is not a threat in our study.

7.2.2.8. Lack of real-world problem instances. Again, this problem is not a threat to the current study because the QoS time series used in our experiments stems from a QoS dataset that gathered and recorded original, real-world QoS values with no modifications (raw data). We test both the proposed approach and the compared methods under real-world conditions, making our experimental observations, discussions, and conclusions reliable.

7.2.3. An evaluation of the SBSE validity of our GP-based approach

This section focuses on demonstrating that the proposed GP-based approach is a valid SBSE study and instance. We rely on the evaluation criteria suggested by the authors who proposed SBSE in [23] to verify our study.

7.2.3.1. Better than random search. In the original content of [23], the authors stated that “*any search-based technique must perform better than a purely random search to qualify as worthy of consideration as a successful application of SBSE*”. Generally, random search provides a baseline criterion for the application of heuristic search (GP). As introduced in Section 6.5 and discussed in the third conclusion validity threat “*lack of a meaningful comparison baseline*”, the compared methods in this paper include a random search, and the results of our GP-based approach are substantially better than random search in forecasting performance (accuracy), as shown in Section 7.1.4 and discussed in Section 7.2.1.

7.2.3.2. Discovery of desirable solutions. Because a fitness function is a measure of the goodness of fit of the solutions (chromosomes) to a specific problem, the authors of [23] considered that a natural approach to performing SBSE validation is to evaluate *how much the fitness increases* and then *gather empirical data to provide evidence*. Following this approach, we randomly select one of the GP experiments and display the fitness values from that experiment in Fig. 8. The figure contains two parts. The upper plot shows the fitness value of the best chromosome (predictor) in each generation.

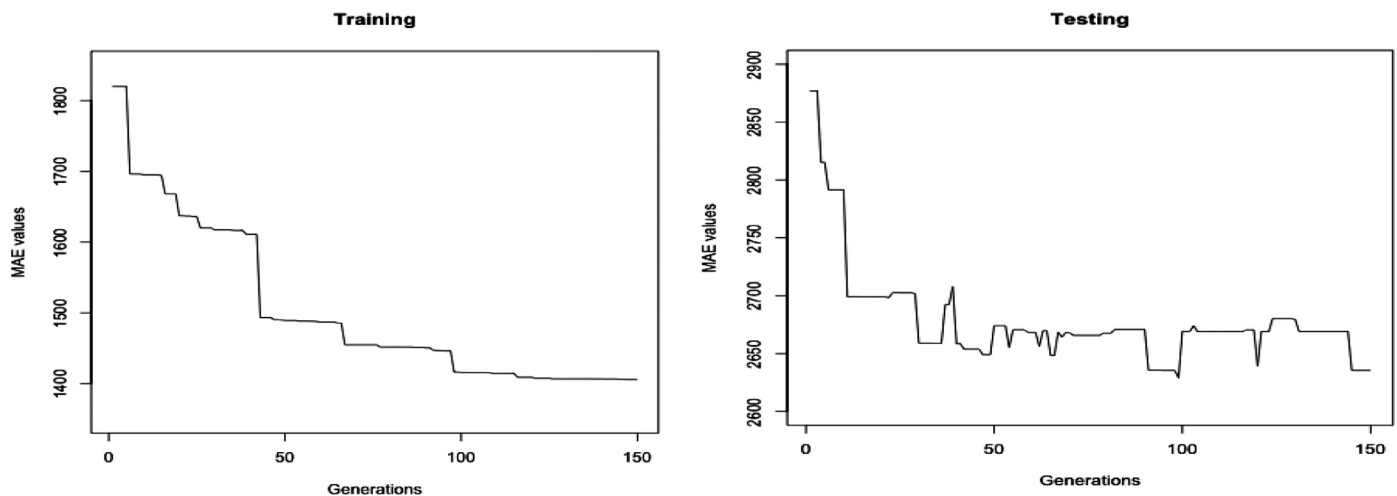


Fig. 8. The variation of the fitness value (training) and forecasting accuracy (testing) during one of the performed GP evolutions; both are measured using the MAE.

The bottom plot presents the forecasting accuracy of the best predictor in each generation, evaluated by feeding previously unseen testing data to the predictor. The fitness improvement and the enhancement of forecasting accuracy are visible, clear, and definite. Due to space limitations, we cannot show the same type of figure for every GP experiment (their multiple runs), although most exhibit a pattern similar to Fig. 8 (with continuous improvement and enhancement to both the fitness value and forecasting accuracy).

7.2.3.3. Efficiency. This topic was covered in the first paragraph of Section 7.2.2.

7.2.3.4. Avoiding bias in analytical approach. In [23], the phrase *analytical approach* represents any non-search-based solution (algorithm), and in this paper, the compared methods introduced in Section 6 are all analytical approaches (not search-based). In [23], this criterion was intended to focus on the *novelty of solutions relative to those produced by the analytic approaches*. The proposers of the SBSE noted that—even in a situation in which an SBSE approach does not outperform a compared analytical approach—the former approach is still valid in SBSE in terms of the *additional insight gained from considering solutions previously thought atypical or non-standard*. In GP, the solution (predictor) search space is determined by its function and terminal sets, including all possible and permitted combinations (predictors) of the elements (operands and operators) contained in the function and terminal sets. Thus, with a proper configuration of the two sets, the predictors that are searched and tried by GP will be novel compared to those that are allowed and can be produced by analytical approaches (the methods in Section 6). Thus, GP provides additional insight by considering solutions (predictors) that analytical approaches never consider. For example, in Section 7.1.4, we demonstrated a predictor found by our GP-based approach. Obviously, this predictor would never be found using any analytical approach because in the compared methods, the predictor form (the skeleton of expressions) and its allowed variations (different coefficient values) are fixed and restricted, respectively, as discussed in Section 6. For example, in ARIMA models, the generated predictors are always a weighted summation of past observations and/or forecasting errors. The fixations and restrictions of the compared (analytical) methods can be considered to be their forecasting biases and preferences. The GP approach avoids these biases and preferences by considering and covering the entire predictor space.

8. Conclusions

In this paper, we proposed and implemented a *search-based software engineering* technique, i.e., *genetic programming*, to solve a defined one-step-ahead QoS forecasting problem. According to the detailed explanation, analysis, comparisons, and experimental results presented in this paper, the proposed GP-based approach has been fully tested and shown to be the most suitable solution for the one-step-ahead QoS forecasting problem. Using our simulated real-world QoS forecasting contexts (the TSs), compared with the other time series forecasting methods included and compared in this study, the proposed approach was found to have the lowest average forecasting error. In our experiments with different TSs, the best forecasting solution differed among the tested segments; our proposed approach was not always the best solution. However, the common weakness of the compared solutions was that they may perform very well in some TSs while performing poorly in others. In contrast, the forecasting accuracy of our approach remained stable. In TSs where the proposed approach was not the best solution, it was only slightly worse than the best solution. Finally, we also followed the validation recommendations of its proposers to show that our approach is a valid and successful implementation of search-based software engineering.

Acknowledgment

This research is partially sponsored by the Ministry of Science and Technology (Taiwan) under the Grant MOST103-2221-E-030-010-MY2.

References

- [1] Y.-Y. Fanjiang, Y. Syu, S.-P. Ma, J.-Y. Kuo, An overview and classification of service description approaches in automated service composition research, *IEEE Trans. Serv. Comput.* (2015) 1.
- [2] D. Zhovtobryukh, A petri net-based approach for automated goal-driven web service composition, *Simulation* 83 (2007) 33–63.
- [3] A. Michlmayr, F. Rosenberg, P. Leitner, S. Dustdar, End-to-end support for QoS-aware service selection, binding, and mediation in VRESCO, *IEEE Trans. Serv. Comput.* 3 (2010) 193–205.
- [4] L. Qianhui, W. Xindong, L. Hoong Chuin, Optimizing service systems based on application-level QoS, *IEEE Trans. Serv. Comput.* 2 (2009) 108–121.
- [5] Z. Obrenovic, D. Gasevic, End-user service computing: spreadsheets as a service composition tool, *IEEE Trans. Serv. Comput.* 1 (2008) 229–242.
- [6] A. Alves, A. Arkin, S. Askary, C. Barreto, B. Bloch, F. Curbera, M. Ford, Y. Goland, A. Guizar, N. Kartha, C. Liu, OASIS Web Services Business Process Execution Language (WSBP EL) v2.0, OASIS Standard 2007.
- [7] Q.Z. Sheng, X. Qiao, A.V. Vasilakos, C. Szabo, S. Bourne, X. Xu, Web services composition: a decade's overview, *Inf. Sci.* 280 (2014) 218–238.

- [8] Y.-Y. Fanjiang, Y. Syu, Semantic-based automatic service composition with functional and non-functional requirements in design time: a genetic algorithm approach, *Inf. Softw. Technol.* 56 (2014) 352–373.
- [9] Z. Li, L. O'Brien, Towards effort estimation for web service compositions using classification matrix, *Int. J. Adv. Internet Technol.* 3 (2010) 245–260.
- [10] Y. Syu, Y.-Y. Fanjiang, A survey to service composition methods using aspects classification, in: 2013 IEEE 7th International Symposium on Service Oriented System Engineering, 2013, pp. 170–181.
- [11] Y. Syu, S.-P. Ma, J.-Y. Kuo, Y.-Y. Fanjiang, A survey on automated service composition methods and related techniques, in: 2012 IEEE Ninth International Conference on Services Computing, 2012, pp. 290–297.
- [12] P. Bartalos, M. Bieliková, Automatic dynamic web service composition: a survey and problem formalization, *Comput. Inf.* 30 (2011) 793–827.
- [13] S. Dustdar, W. Schreiner, A survey on web services composition, *Int. J. Web Grid Serv.* 1 (2005) 1–30.
- [14] J. Rao, X. Su, A survey of automated web service composition methods, in: *Proceedings of the First International Conference on Semantic Web Services and Web Process Composition*, Springer-Verlag, 2005, pp. 43–54.
- [15] X. Han, Y. Liu, B. Xu, G. Zhang, A survey on QoS-aware dynamic web service selection, in: 2011 7th International Conference on Wireless Communications, Networking and Mobile Computing, 2011, pp. 1–5.
- [16] M. Rambold, H. Kasinger, F. Lautenbacher, B. Bauer, Towards autonomic service discovery a survey and comparison, in: *IEEE International Conference on Services Computing*, 2009, pp. 192–201.
- [17] A. Moraru, C. Fortuna, B. Fortuna, R.R. Slavescu, A hybrid approach to QoS-aware web service classification and recommendation, in: *IEEE 5th International Conference on Intelligent Computer Communication and Processing*, 2009, pp. 343–346.
- [18] K. Kritikos, D. Plexousakis, Requirements for QoS-based web service description and discovery, *IEEE Trans. Serv. Comput.* 2 (2009) 320–337.
- [19] M.H. Zadeh, M.A. Seyyedi, Qos monitoring for web services by time series forecasting, in: 2010 3rd IEEE International Conference on Computer Science and Information Technology, 2010, pp. 659–663.
- [20] M. Harman, S.A. Mansouri, Y. Zhang, Search-based software engineering: trends, techniques and applications, *ACM Comput. Surv.* 45 (2012) 1–61.
- [21] N. Wagner, Z. Michalewicz, M. Khouja, R.R. McGregor, Time series forecasting for dynamic environments: the DyFor genetic program model, *IEEE Trans. Evolut. Comput.* 11 (2007) 433–452.
- [22] S. Yang, F. Yong-Yi, K. Jong-Yih, M. Shang-Pin, Applying genetic programming for time-aware dynamic QoS prediction, in: 2015 IEEE International Conference on Mobile Services, 2015, pp. 217–224.
- [23] M. Harman, B.F. Jones, Search-based software engineering, *Inf. Softw. Technol.* 43 (2001) 833–839.
- [24] T. Senivongse, N. Wongsawangpanich, Composing services of different granularity and varying QoS using genetic algorithm, in: *Lecture Notes in Engineering and Computer Science: Proceedings of The World Congress on Engineering and Computer Science*, 2011, pp. 388–393.
- [25] Z. Ye, S.K. Mistry, A. Bouguettaya, H. Dong, Long-term QoS-aware cloud service composition using multivariate time series analysis, *IEEE Trans. Serv. Comput.* (2014) 1–1.
- [26] Z. Zibin, Z. Yilei, M.R. Lyu, Distributed QoS evaluation for real-world web services, in: 2010 IEEE International Conference on Web Services, 2010, pp. 83–90.
- [27] Z. Zheng, Y. Zhang, M. Lyu, Investigating QoS of real-world web services, *IEEE Trans. Serv. Comput.* (2012) 1.
- [28] Z. Zheng, M.R. Lyu, Collaborative reliability prediction of service-oriented systems, in: *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering*, 1, 2010, pp. 35–44.
- [29] Z. Zibin, M. Hao, M.R. Lyu, I. King, Collaborative web service QoS prediction via neighborhood integrated matrix factorization, *IEEE T. Serv. Comput.* 6 (2013) 289–299.
- [30] Z. Zibin, M. Hao, M.R. Lyu, I. King, QoS-aware web service recommendation by collaborative filtering, *IEEE Trans. Serv. Comput.* 4 (2011) 140–152.
- [31] Z. Zheng, M.R. Lyu, Personalized reliability prediction of web services, *ACM Trans. Softw. Eng. Methodol.* 22 (2013) 1–25.
- [32] B. Cavallo, M.D. Penta, G. Canfora, An empirical comparison of methods to support QoS-aware service selection, in: *Proceedings of the 2nd International Workshop on Principles of Engineering Service-Oriented Systems*, Cape Town, South Africa, ACM, 2010, pp. 64–70.
- [33] A. Amin, A. Colman, L. Grunske, An approach to forecasting QoS attributes of web services based on ARIMA and GARCH models, in: 2012 IEEE 19th International Conference on Web Services, 2012, pp. 74–81.
- [34] A. Amin, L. Grunske, A. Colman, An automated approach to forecasting QoS attributes based on linear and non-linear time series modeling, in: *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering*, 2012, pp. 130–139.
- [35] M. Godse, U. Bellur, R. Sonar, Automating QoS based service selection, in: 2010 IEEE International Conference on Web Services, 2010, pp. 534–541.
- [36] X. YunNi, D. Jian, L. Xin, Z. QingSheng, Dependability prediction of WS-BPEL service compositions using petri net and time series models, in: 2013 IEEE 7th International Symposium on Service Oriented System Engineering, 2013, pp. 192–202.
- [37] Z. Yilei, Z. Zibin, M.R. Lyu, WSPred: a time-aware personalized QoS prediction framework for web services, in: 2011 IEEE 22nd International Symposium on Software Reliability Engineering, 2011, pp. 210–219.
- [38] H.Q. Yu, S. Reiff-Marganiec, Non-functional property based service selection: a survey and classification of approaches, *The 6th IEEE European Conference on Web Services, Non Functional Properties and Service Level Agreements in Service Oriented Computing Workshop*, IEEE, 2008.
- [39] W. Lijuan, S. Jun, Y. Jianming, A survey on bio-inspired algorithms for web service composition, in: 2012 IEEE 16th International Conference on Computer Supported Cooperative Work in Design, 2012, pp. 569–574.
- [40] Y. Shi, X. Chen, A survey on QoS-aware web service composition, in: 2011 Third International Conference on Multimedia Information Networking and Security, 2011, pp. 283–287.
- [41] C.-F. Lin, R.-K. Sheu, Y.-S. Chang, S.-M. Yuan, A relaxable service selection algorithm for QoS-based web service composition, *Inf. Softw. Technol.* 53 (2011) 1370–1381.
- [42] G. Zou, Q. Lu, Y. Chen, R. Huang, Y. Xu, Y. Xiang, QoS-aware dynamic composition of web services using numerical temporal planning, *IEEE T. Serv. Comput.* (2012) 5.
- [43] J. Wei, H. Songlin, L. Zhiyong, Top *k* query for QoS-aware automatic service composition, *IEEE Trans. Serv. Comput.* 7 (2014) 681–695.
- [44] F. Wagner, F. Ishikawa, S. Honiden, QoS-aware automatic service composition by applying functional clustering, in: 2011 IEEE International Conference on Web Services, 2011, pp. 89–96.
- [45] L. Mu, H. Jinpeng, G. Huipeng, An adaptive web services selection method based on the QoS prediction mechanism, in: *IEEE/WIC/ACM International Joint Conferences on Web Intelligence and Intelligent Agent Technologies*, 2009, pp. 395–402.
- [46] R.J. Hyndman, G. Athanasopoulos, *Forecasting: Principles and Practice*, OTexts, 2014.
- [47] T. Mitchell, *Machine Learning*, McGraw Hill, Boston, MA, 1997.
- [48] C.J. Burgess, M. Lefley, Can genetic programming improve software effort estimation? A comparative evaluation, *Inf. Softw. Technol.* 43 (2001) 863–873.
- [49] M. Santini, A. Tettamanzi, Genetic programming for financial time series prediction, in: J. Miller, M. Tomassini, P. Lanzi, C. Ryan, A. Tettamanzi, W. Langdon (Eds.), *Genetic Programming*, Springer, Berlin Heidelberg, 2001, pp. 361–370.
- [50] H. Iba, T. Sasaki, Using genetic programming to predict financial data, in: *Proceedings of the 1999 Congress on Evolutionary Computation*, 1999, p. 251.
- [51] A. Agapitos, M. Dyson, J. Kovalchuk, S.M. Lucas, On the genetic programming of time-series predictors for supply chain management, in: *Proceedings of the 10th Annual Conference on Genetic and Evolutionary Computation*, ACM, 2008, pp. 1163–1170.
- [52] K. Meffert, JGAP - java genetic algorithms and genetic programming package, Available at: <http://jgap.sf.net>.
- [53] R.J. Hyndman, *Forecasting functions for time series and linear models* Available at:.
- [54] M.d.O. Barros, A.C.D. Neto, *Threats to Validity in Search-based Software Engineering Empirical Studies*, UNIRIO - Universidade Federal do Estado do Rio de Janeiro, Rio de Janeiro, 2011.